



**ХИМИКОТЕХНОЛОГИЧЕН И МЕТАЛУРГИЧЕН УНИВЕРСИТЕТ
СОФИЯ
ФАКУЛТЕТ ПО ХИМИЧНО И СИСТЕМНО ИНЖЕНЕРСТВО
КАТЕДРА „ АВТОМАТИЗАЦИЯ НА ПРОИЗВОДСТВОТО “**

ДИПЛОМНА РАБОТА

**ТЕМА: „ СПЕЦИФИКАЦИЯ НА СОФТУЕРНИТЕ ИЗИСКВАНИЯ НА
СИСТЕМА ЗА ОНЛАЙН КОНСУЛТАЦИИ С МЕДИЦИНСКИ
СПЕЦИАЛИСТИ “**

ОБРАЗОВАТЕЛНО КВАЛИФИКАЦИОННА СТЕПЕН: БАКАЛАВЪР

Ръководител на катедра:

/доц. д-р инж. Даниела Гочева/

Научен ръководител:

/гл. ас. д-р инж. Васил Методиев/

Дипломант:

/Кирил Николаев Кискинов/
Фак. № ИИ0014

**София,
юли, 2024 год.**



UNIVERSITY OF CHEMICAL TECHNOLOGY AND METALLURGY - SOFIA
FACULTY OF CHEMICAL AND SYSTEM ENGINEERING
DEPARTMENT OF INDUSTRIAL AUTOMATION

Specialty: Informatics Engineering

DIPLOMA THESIS

Title:

**SPECIFICATION OF SOFTWARE REQUIREMENTS FOR AN ONLINE
CONSULTATION SYSTEM WITH MEDICAL SPECIALISTS**

Head of Department:

/Assoc. Prof. Dr. Eng. D. Gocheva /

Supervisor:

/Chief Assist. Prof. Dr. Eng. Vasil Metodiev/

Student:

/Kiril Kiskinov F.Nº ИИ0014/

Sofia

2024

**ХИМИКОТЕХНОЛОГИЧЕН И МЕТАЛУРГИЧЕН УНИВЕРСИТЕТ
ФАКУЛТЕТ ПО СИСТЕМНО И ХИМИЧНО ИНЖЕНЕРСТВО**

Катедра: “АВТОМАТИЗАЦИЯ НА ПРОИЗВОДСТВОТО”

Специалност: *“Инженерна информатика ”*

Образователно-квалификационна степен: БАКАЛАВЪР

Утвърдил:

Ръководител на катедра АП:.....

/доц. д-р Даниела Гочева/

**ЗАДАНИЕ
ЗА ДИПЛОМНА РАБОТА**

на студента Кирил Николаев Кискинов, фак.№ ИИ0014

Тема на дипломната работа

**„Спецификация на софтуерните изисквания на система за онлайн консултации
с медицински специалисти“**

1. Да се проучат моделите за анализ, спецификациите на хардуер, софтуер и интерфейсни спецификации, тестването на функционалности и сигурност на системите и изграждането на потребителски интерфейси за системи за онлайн спецификации.
2. Да се проектира система за онлайн консултации с медицински специалисти.
3. Да се приложи подходящ потребителски интерфейс и да се проведат всички необходими тестове.

Дата на получаване

на заданието: 27.05.2024г.

Студент:.....

/Кирил Кискинов, ИИ0014/

Срок за предаване

на дипломната работа: 17.07.2024г.

Ръководител:.....

/гл.ас.д-р инж. Васил Методиев/

ХИМИКОТЕХНОЛОГИЧЕН И МЕТАЛУРГИЧЕН УНИВЕРСИТЕТ

СОФИЯ

ДЕКЛАРАЦИЯ ЗА АВТОРСТВО¹

Име _____

Факултетен № _____

Катедра: _____

Факултет: _____

Заглавие на дипломната работа:

С настоящето потвърждавам, че целият материал е разработен от мен, с изключение на специално посочените пасажки, и че:

- ☒ Ясно съм посочил/а използваните източници в текста и библиографската справка
- ☒ Посочил/а съм източниците на всички снимки, таблици, графики и др. които не са резултат на моя труд;
- ☒ Не съм използвал/а курсови работи, доклади, реферати, есета, на настоящи и бивши студенти в обем, по-голям от 10% от общия обем на материала;
- ☒ Спазвал/а съм избрана система за цитиране в целия обем на разработката;
- ☒ Съобразил/а съм се със всички критерии и изисквания за недопускане на плагиатство, посочени в общоприетите критерии, като претенция за авторство на произведение, което в действителност е създадено от друг.

Всяко мое невярно твърдение в тази декларация ще бъде санкционирано в съответствие с нормативната уредба на ХТМУ.

Подпис:

Дата:
София

¹ Декларацията трябва да бъде попълнена от студента, като се отметне всяко твърдение, за да е ясно, че съответното условие е изпълнено. Подписаната и датирана декларация трябва да бъде част от приложените към дипломната работа документи..

Съдържание

1.	Увод	3
2.	Литературен обзор	4
2.1.	Историческо развитие на телемедицината и онлайн консултациите	4
2.2.	Текущо състояние на телемедицината	5
2.3.	Модели на жизнения цикъл на софтуерния процес	6
2.4.	Спецификация на софтуерните изисквания	17
2.5.	Техники за събиране на изисквания	19
2.6.	Регулаторни изисквания	20
2.7.	Технологии за защита на данните	21
2.8.	Потребителски интерфейс и използваемост:	23
2.9.	Методи за тестване на използваемост	25
2.10.	Интеграция и взаимодействие с други системи	27
2.11.	Изводи от литературния обзор	29
3.	Експериментална част	31
3.1.	Обекти на изследване.....	31
3.1.1.	Цел на проекта.....	31
3.1.2.	Предвидена аудитория	31
3.1.3.	Обхват на проекта	31
3.1.4.	Обзор на документа	32
3.1.5.	Гледна точка на продукта	33
3.1.6.	Характеристики на продукта	33
3.1.7.	Ограничения на дизайна и изпълнението	35
3.1.8.	Предположения и зависимости.....	35
3.1.9.	Съществуващи проблеми, които системата решава.....	35
3.1.10.	Системни изисквания	36
3.1.10.1.	Функционални изисквания.....	36
3.1.10.2.	Нефункционални изисквания.....	38
3.1.11.	Архитектура на системата.....	41
3.1.11.1.	Архитектура Model-View-Controller (MVC)	41
3.1.11.2.	Системни хардуерни изисквания.....	42
3.1.11.3.	Системни софтуерни изисквания:	42
3.2.	Методи за изследване	43
3.2.1.	Езикът за програмиране Java	43

3.2.2.	Избор на система за управление на бази данни	45
3.2.3.	Анализ на изискванията чрез диаграми	47
3.2.4.	Методи за верификация и валидация	50
4.	Обсъждане на експерименталните резултати, графики и таблици	53
4.1.	Подробна спецификация на случаите на употреба	53
4.2.	Модели за анализ	62
4.2.1.	Диаграма на случаите на използване (Use case diagram)	62
4.2.2.	Диаграма на класовете (Class Diagram)	64
4.2.3.	ER Diagram (Entity–relationship model)	67
4.2.4.	Диаграма на потока от данни (Data Flow Diagram)	69
4.2.5.	Диаграма на последователност (Sequence Diagram)	70
4.3.	Приемни тестови случаи (Acceptance Test Cases)	72
4.3.1.	Тест за приемане при влизане (Login Acceptance Test)	72
4.3.2.	Тест за приемане на регистрация (Registration Acceptance Test)	73
4.3.3.	Тест за приемане на началната страница (Home Page Acceptance Test)	74
4.3.4.	Тест за приемане на резервация (Book Appointment Acceptance Test)	74
4.3.5.	Тест за приемане на потребителски профил (Account Profile Acceptance Test)	75
4.3.6.	Тест за приемане на пациента (Patient Acceptance Test)	76
4.3.7.	Тест за приемане на час за среща (Appointment Acceptance Test)	77
4.3.8.	Тест за приемане на запитванията (Make Enquiries Acceptance Test)	78
4.4.	Примерен потребителски интерфейс	79
5.	Заклучение	85
	Литература	87

1. Увод

В последните години телемедицината и онлайн консултациите с медицински специалисти стават все по-значима част от здравеопазването [1]. С развитието на технологиите и необходимостта от достъп до медицински услуги независимо от местоположението на пациента, онлайн консултациите предоставят нови възможности за подобряване на здравната грижа. Тези системи позволяват на пациентите да получат бърз и удобен достъп до медицински специалисти, като същевременно намаляват разходите и времето, свързани с традиционните медицински посещения.

Въпреки значителния потенциал, разработването на системи за онлайн консултации с медицински специалисти изисква прецизно дефиниране на софтуерните изисквания. Това включва не само функционалните аспекти на системата, но и изискванията за сигурност, конфиденциалност, използваемост и съответствие с регулаторните стандарти.

Основната цел на тази дипломна работа е да се разработи спецификация на софтуерните изисквания за система за онлайн консултации с медицински специалисти, която да отговаря на нуждите на потребителите и да спазва всички съответни регулаторни изисквания.

За постигане на целта бяха поставени за решаване **следните задачи**:

- 1. Провеждане на литературен обзор** на съществуващите изследвания и разработки в областта на изграждането на онлайн системи.
- 2. Изследване на съществуващите регулации** и стандарти за сигурност и защита на медицинските данни.
- 2. Определяне на потребителските изисквания:** Събиране и анализ на нуждите и очакванията на потребителите (пациенти и медицински специалисти).
- 3. Дефиниране на функционалните изисквания:** Определяне на необходимите функционалности на системата, включително регистрация на потребители, записване за консултация, провеждане на видео консултации и управление на електронни рецепти.
- 4. Дефиниране на нефункционалните изисквания:** Определяне на изискванията за сигурност, конфиденциалност, производителност и използваемост на системата.
- 5. Разработване на техническа архитектура:** Създаване на архитектурен дизайн на системата, включително избор на технологии и платформи, и дефиниране на интерфейсите за интеграция с други системи.
- 6. Верификация и валидация:** Провеждане на тестове за верификация и валидация на изискванията и функционалностите на системата.

2. Литературен обзор

2.1. Историческо развитие на телемедицината и онлайн консултациите

Телемедицината е област, която се развива значително през последните десетилетия, осигурявайки дистанционен достъп до медицински услуги [1]. Историческото развитие на телемедицината започва с използването на телефонни консултации и радиовръзки за предоставяне на медицинска помощ в отдалечени райони. С развитието на интернет технологиите и мобилните комуникации, телемедицината еволюира до използване на видеоконференции и мобилни приложения, които позволяват по-ефективно и удобно взаимодействие между пациенти и медицински специалисти.

Идеята за телемедицина възниква още в 1920-те години с радиотерапевтични консултации [2]. През 1950-те и 1960-те години различни изследователи и учени използват видео и аудио технологии за дистанционно медицинско обслужване. През 1990-те и по-късно, с развитието на интернет и цифровите комуникации, телемедицината започва да се разраства значително. Лесният достъп до високоскоростни интернет връзки и подобрените компютърни системи правят възможно предоставянето на онлайн консултации в реално време.

Телемедицината намира широко приложение в различни области на медицината, като дерматология, психиатрия, хронични заболявания и т.н. Пациентите могат да получават медицински съвети, изследвания и следене на състоянието си без физическо присъствие в клиниката.

С разширяването на телемедицинските услуги се появяват и нови регулаторни и етични предизвикателства [3]. Различните страни имат различни стандарти и правила за употреба на тези технологии. Това включва въпроси като лицензиране на здравни специалисти за предоставяне на телемедицински услуги, защита на данните на пациентите и други правни аспекти. Тъй като телемедицината изисква обмен на медицински и лични данни между пациенти и медицински специалисти през цифрови канали, въпросът за защита на данните и поверителността е критичен. Регулаторните рамки често включват изисквания за сигурност на данните и съответствие със законодателството за защита на личните данни.

Етичните стандарти включват въпроси като етика на дистанционната медицина, доверие и професионализъм. Медицинските съвети и процедури трябва да се извършват в съответствие с високи стандарти за качество на грижите и етично поведение на здравните

работници. С растежа на телемедицината се появяват нови медико-правни въпроси, свързани с отговорността на медицинските специалисти при предоставяне на услуги дистанционно. Това включва въпроси за диагностика и лечение, както и за комуникация и консултации с пациентите.

2.2. Текущо състояние на телемедицината

През последното десетилетие, онлайн медицинските консултации стават все по-популярни, особено с оглед на глобалната пандемия от COVID-19, която ускорява внедряването на телемедицински решения [4]. Текущите изследвания в областта фокусират върху разработката на системи, които осигуряват високо ниво на сигурност, конфиденциалност и използваемост. Иновациите включват използването на изкуствен интелект за автоматизиране на определени процеси, както и интеграцията на телемедицинските системи с електронни здравни досиета и други здравни информационни системи.

В условия на пандемия телемедицината предоставя безопасен начин за пациентите да получат консултации и медицинска помощ, без да се излагат на риск от зараза в клинични среди. Това е от особено значение за хора с висок риск или хронични заболявания.

За времето на COVID-19 телемедицината е била широко разпространена за диагностика, консултации, следене на симптоми и психологическа подкрепа на пациентите. Включването на технологии като видео конферентни платформи, смартфони и онлайн платформи за консултации е улеснило достъпа до здравеопазване, особено в области с ограничени ресурси.

Телемедицината е подпомогнала здравните работници да координират грижите за пациентите и да ограничат разпространението на инфекцията. Здравните специалисти могат да работят от разстояние и да предоставят съвети и инструкции на колегите и пациентите.

Въпреки успехите COVID-19 изложи и предизвикателства като нуждата от бързо развитие на телемедицински технологии и обучение на персонала. От друга страна, това ускори приемането на телемедицината и поощри инвестициите в технологии и инфраструктура.

След пандемията от COVID-19 телемедицината продължава да играе важна роля в трансформацията на здравеопазването. Много здравни системи по света осъзнаха ползите от телемедицината и започнаха да я интегрират в ежедневната практика. Очаква се телемедицината да остане неразделна част от здравната грижа, предлагайки удобство и достъпност за пациентите.

2.3. Модели на жизнения цикъл на софтуерния процес

Жизненият цикъл на разработка на софтуер (Software Development Life Cycle, SDLC) представлява процесът, чрез който се планира, създава, тества и поддържа софтуерен продукт. Той включва няколко основни етапа [5]:

1. Планиране (Planning)

Цел: Определяне на обхвата и целите на проекта.

Дейности: Събиране на изисквания, анализ на ресурси, изготвяне на времеви график и бюджет.

2. Анализ на изискванията (Requirements Analysis)

Цел: Разбиране на нуждите на потребителите и изискванията към системата.

Дейности: Интервюта с потребители, анализ на бизнес процеси, изготвяне на документи за изисквания.

3. Проектиране (Design)

Цел: Разработка на архитектурата и дизайна на системата.

Дейности: Създаване на функционален дизайн, избор на технологии и инструменти, изготвяне на диаграми и прототипи.

4. Разработка (Development)

Цел: Написване на код и изграждане на софтуера.

Дейности: Програмиране, интеграция на компоненти, верификация на кода.

5. Тестване (Testing)

Цел: Проверка за грешки и удостоверяване, че софтуерът отговаря на изискванията.

Дейности: Извършване на различни видове тестове (модулни, интеграционни, системни, приемателни), отстраняване на грешки.

6. Инсталиране и внедряване (Deployment)

Цел: Инсталиране на софтуера в производствена среда и предаване на потребителите.

Дейности: Подготовка на производствена среда, миграция на данни, обучение на потребители, пускане на софтуера в експлоатация.

7. Поддръжка и обновяване (Maintenance and Updates)

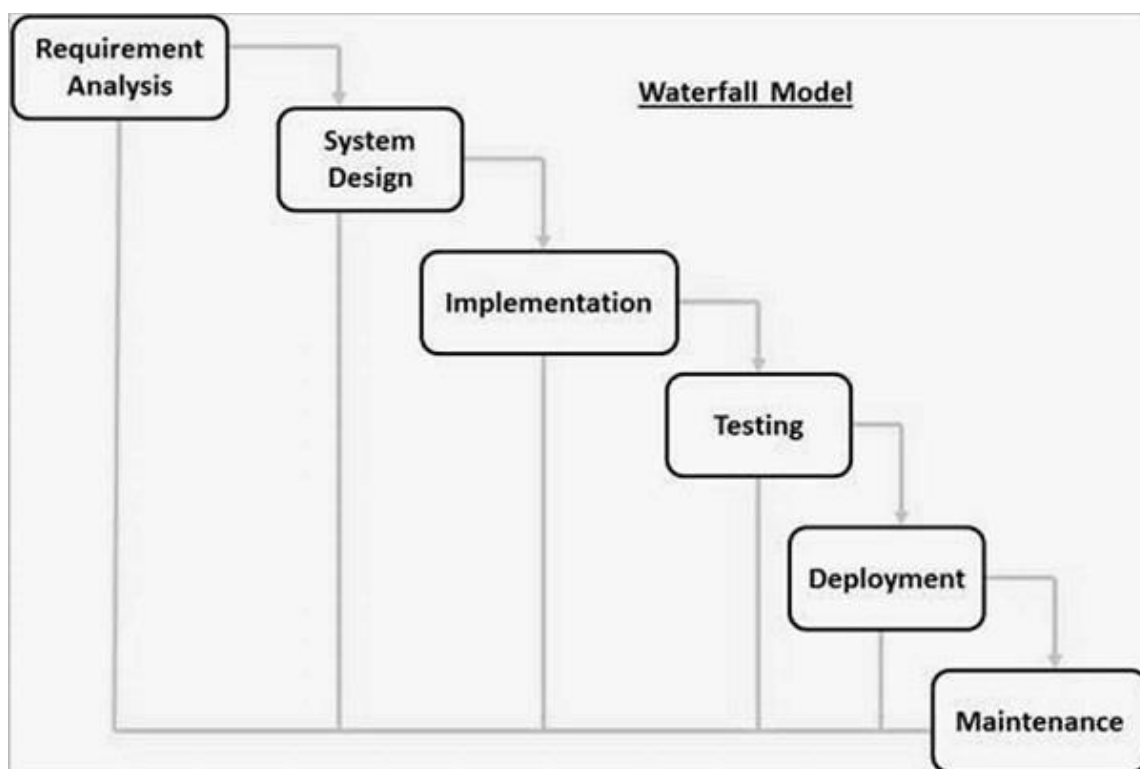
Цел: Осигуряване на дългосрочна работа на софтуера и обновяване при нужда.

Дейности: Поправка на грешки, добавяне на нови функции, оптимизация на производителността, актуализация на документация.

Моделите за разработване на софтуерен продукт представляват различни методологии и подходи, които се използват за организиране, планиране и изпълнение на процеса на софтуерно развитие. Съществуват няколко основни вида модели за разработка на софтуерен продукт, като те се делят на **традиционни** (Модел на водопада, Прототипен модел, Спирален модел, V – образен модел) и **гъвкави** (Scrum, FDD - разработване по функционалности, Kanban). Всеки модел има своите специфики, които го правят подходящ за различни типове проекти. Някои от основните модели за разработване на софтуерни продукти са:

А) Традиционни:

Модел на водопада (Waterfall Model)



Фиг. 1 Схема на модела на водопада [6]

Водопадният модел е линеен и последователен подход, при който всяка фаза от разработката трябва да бъде завършена преди преминаване към следващата фаза.

Фази:

- **Изисквания:** Събиране и анализ на изискванията.
- **Дизайн:** Планиране и проектиране на архитектурата на системата.
- **Имплементация:** Кодиране на системата.
- **Тестване:** Проверка и валидация на системата.
- **Инсталация:** Деплоймънт на системата в производствена среда.
- **Поддръжка:** Поддръжка и актуализация на системата.

Предимства:

- Отделните фази са ясно дефинирани и разграничени.
- Лесно се управляват бюджета, времевия график и документацията.

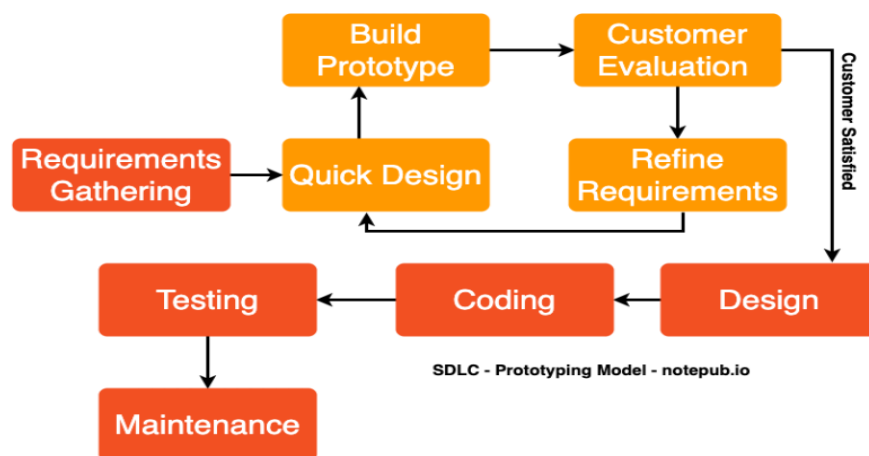
Недостатъци:

- Грешка, която е допусната в началните фази се пренася във всяка следваща фаза.
- Висока степен на риск и несигурност – работещ софтуер се получава в последната фаза.

Използва се:

- При проекти, които имат ясно определени изисквания и които няма да се променят във времето.
- Работи добре при малки и кратки за изпълнение проекти.

Прототипен модел (Prototyping Model)



Фиг. 2 Схема на прототипен модел [7]

Прототипният модел включва създаване на прототипи на системата, които се разработват и тестват, за да се разберат и уточнят изискванията.

Фази:

- **Събиране на изисквания:** Основни изисквания са събрани и разбрани.
- **Бързо разработване на прототип:** Създаване на начална версия на системата.
- **Оценка на прототипа:** Обратна връзка от потребителите и идентифициране на недостатъци.
- **Преработка:** Актуализиране на изискванията и подобрене на прототипа.
- **Разработка и тест на продукта:** Разработка на финалната система, базирана на утвърдените изисквания.

Предимства:

- Кратък цикъл на разработка.
- Наличието на прототип в края на всяка итерация.
- Навременна обратна връзка от клиента, по-лесното събиране и разбиране на изискванията.

Недостатъци:

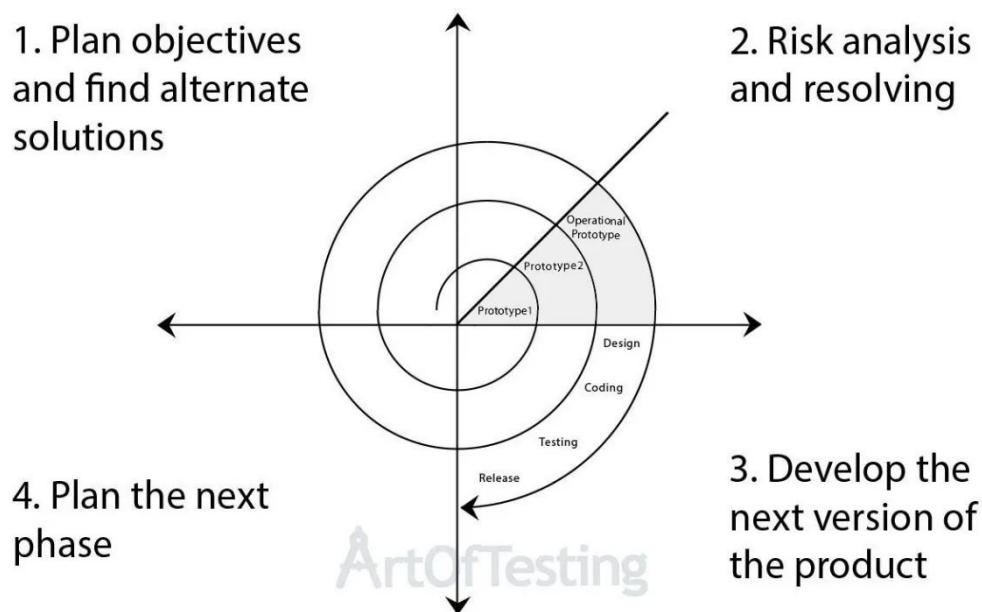
- Методът може да увеличи сложността на системата, обхвата на проекта да се увеличи извън зададения.
- Съкратеното време за разработка на прототипа, фазата на анализ и дизайн води до рискове

Използва се:

- При развитие на еволюционни модели
- Web базирани приложения, при които ще е нужно минимално обучение на крайния потребител

Спирален модел (Spiral Model)

Спиралният модел комбинира итеративния подход с управлението на риска, като всяка итерация включва четири основни фази: планиране, анализ на риска, разработка и оценка.



Фиг. 3 Схема на спирален модел [8]

Фази:

- **Планиране:** Определяне на целите и изискванията за всяка итерация.
- **Анализ на риска:** Идентифициране и управление на потенциални рискове.
- **Инженерство:** Разработка и тестване на системата.
- **Оценка:** Оценка на резултатите и планиране на следващата итерация.

Предимства:

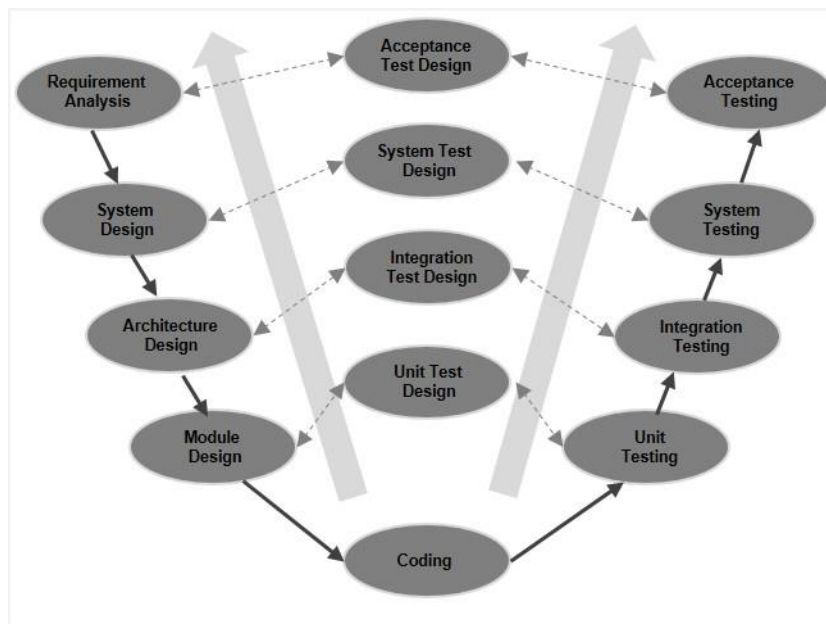
- Може да се добавят допълнителни функционалности.
- Подходящ е за големи проекти.
- Има работещ прототип след всяка итерация.

Недостатъци:

- Не е подходящ за малки проекти поради необосновано ангажиране на ресурси.
- Успеха на проекта е силно зависим от фазата на анализ на риска.

Използва се:

- Проекти, при които изискванията са сложни и потребителите не са сигурни в техните нужди.
- При създаване на нов продукт или когато се налагат големи промени.

V-образен модел (V-Model)

Фиг. 4 Схема на V-образен модел [9]

V-образният модел е разширение на водопадния модел, като поставя акцент върху тестването на всяка фаза от разработката. Тестовите дейности са ясно дефинирани и съпоставени с всяка фаза на разработката.

Фази:

- **Изисквания:** Събиране и анализ на изискванията.
- **Системен дизайн:** Проектиране на системата.
- **Архитектурен дизайн:** Детайлизиране на архитектурата.
- **Модулен дизайн:** Дизайн на отделни модули.

- **Имплементация:** Кодиране.
- **Модулно тестване:** Тестване на отделни модули.
- **Интеграционно тестване:** Тестване на взаимодействието между модулите.
- **Системно тестване:** Тестване на цялостната система.
- **Приемно тестване:** Тестване от потребителя.

Предимства:

- Методът е лесен за прилагане.
- Резултатите постигнати след всяка фаза са лесни за проследяване
- Грешки може да се открият в ранните фази.

Недостатъци:

- Не се създава прототип и има висок риск да не се удовлетворят първоначално зададените изискванията.
- Ако се наложат промени цялата документация трябва да се обнови.

Използва се:

- В малки до среден размер проекти
- Когато има ясно дефинирани изисквания за софтуера.

Б) Гъвкави (Agile):

Scrum модел:



Фиг. 5 Схема на Scrum модела [10]

Scrum е гъвкав метод за управление на проекти, който се фокусира върху итеративна и инкрементална разработка, с акцент върху екипната работа и гъвкавостта.

Фази (Спринтове):

- **Планиране на спринт:** Определяне на задачи за кратък период (спринт).
- **Спринт:** Изпълнение на задачите в рамките на 1-4 седмици.
- **Ежедневни срещи:** Кратки срещи за обсъждане на напредъка и проблемите.
- **Преглед на спринт:** Преглед и демонстрация на постигнатото.
- **Ретроспекция:** Оценка и подобрене на процеса.

Предимства:

- Висока гъвкавост и адаптивност.
- Редовна обратна връзка от клиента.
- Ефективно управление на промените.

Недостатъци:

- Изисква активно участие и добра комуникация в екипа.
- Може да бъде предизвикателство за управление на големи проекти.

Ключови роли в Scrum:

1. Продуктов собственик (Product Owner)

Описание: Представител на клиента или заинтересованите страни, който отговаря за максимизирането на стойността на продукта, създаван от екипа.

Отговорности:

- Създаване и управление на продуктовия беклог (Product Backlog).
- Приоритизиране на задачите в беклога спрямо бизнес стойността и нуждите на потребителите.
- Комуникация със заинтересованите страни, за да се гарантира, че екипът работи по най-важните и ценни задачи.
- Осигуряване на яснота и разбиране на изискванията за екипа.

2. Scrum мастер (Scrum Master)

Описание: Уредник и лидер на екипа, който гарантира, че Scrum процесите се следват и подпомага екипа да работи ефективно.

Отговорности:

- Премахване на препятствия и проблеми, които пречат на екипа да изпълни задачите
- Подпомагане на екипа в самоорганизацията и подобряване на процесите.
- Обучение на екипа и организацията за Scrum принципите и практиките.
- Организиране и уреждане на Scrum срещите (ежедневни срещи, планиране на спринт, преглед на спринт, ретроспекция).

3. Разработващ екип (Development Team)

Описание: Група от професионалисти, които имат за задача да изпълнят всички дейности, свързани с разработката на софтуера. Те са съставени от крос-функционален екип, което означава, че включват всички необходими умения за създаване на завършен продукт или инкремент от продукта в рамките на всеки спринт.

Отговорности:

- Самостоятелно управление на задачите и определяне на подхода за изпълнение на работата.
- Изпълнение на задачите, включени в спринта, и доставяне на работещ софтуер.
- Сътрудничество и комуникация с продуктовия собственик и Scrum мастера за постигане на целите на спринта.
- Участие в планирането и оценката на задачите, за да се осигури качествено и навременно изпълнение.

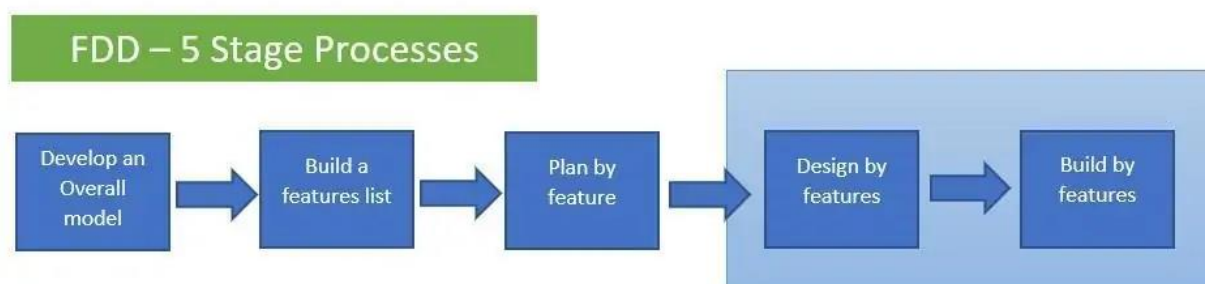
Инкрементът е потенциално доставим продукт, който представлява съвкупност от всички задачи и функции, завършени по време на текущия спринт, както и всички предходни спринтове. Всеки инкремент трябва да бъде напълно функционален и готов за употреба или доставка на клиента, дори ако не включва всички планирани функции на крайния продукт.

Sprint Backlog (беклог на спринта) е списък със задачи и дейности, които екипът по разработка планира да завърши в рамките на текущия спринт. Този списък е подмножество от по-големия Product Backlog (продуктов беклог) и се създава по време на планирането на спринта. Sprint Backlog е ключов инструмент в Scrum процеса, който помага на екипа по разработка да се фокусира върху конкретни и постижими цели, осигурява прозрачност и улеснява управлението и изпълнението на задачите.

Product Backlog (продуктовият беклог) е основен елемент в Scrum методологията. Това е динамичен списък с всички функции, подобрения, корекции и други елементи на

работа, които са необходими за развитието и поддържането на продукта. Продуктивният беклог е постоянно развиващ се документ, който се променя и адаптира през целия жизнен цикъл на проекта, отразявайки нови изисквания, обратна връзка от потребители и други променливи фактори. Product Backlog е основен инструмент за управление на проекта в Scrum, който помага на екипа и заинтересованите страни да поддържат яснота, фокус и приоритети в процеса на разработка.

FDD (Feature Driven Development) - разработване по функционалности:



Фиг. 6 Схема на FDD модела [11]

FDD е модел за разработка на софтуер, който се фокусира върху създаване на функционалности, които са полезни за потребителите. Процесът е разделен на кратки, многократно изпълнявани итерации - до 2 седмици. Методът не покрива целия процес на разработка на софтуер, а се разработва само една функционалност.

Фази:

- **Разбиране на общия модел:** Изграждане на цялостния модел на системата.
- **Създаване на списък с функционалности:** Дефиниране на конкретни функционалности.
- **Планиране по функционалности:** Подреждане и планиране на разработката.
- **Дизайн и разработка по функционалности:** Детайлен дизайн и имплементация на всяка функционалност.
- **Интеграция и тестване:** Обединяване на всички разработени функционалности и тестване на системата.

Предимства:

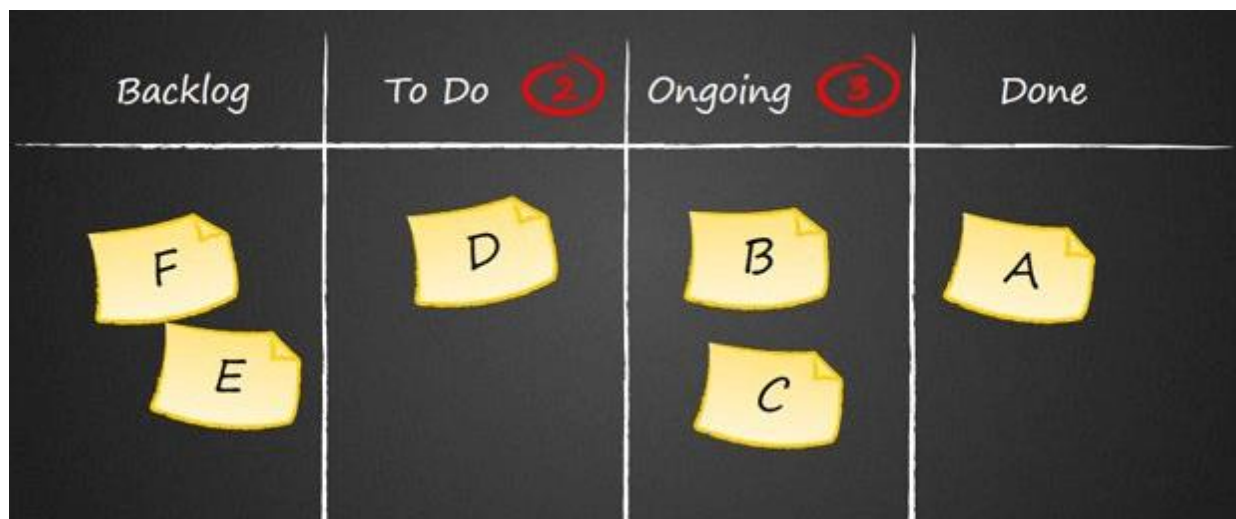
- Фокусиране върху потребителските нужди и функционалностите.

- Ясна структура и лесно проследяване на напредъка.
- Методът е подходящ за големи екипи.

Недостатъци:

- Може да изисква повече време за планиране и координация.
- Не е толкова гъвкав, колкото другите Agile методологии.

Методът Kanban:



Фиг. 7 Схема на Канбан дъска [12]

Kanban е метод за управление на проекти, който използва визуална система за управление на задачи и потоци на работа. Основният акцент е върху непрекъснатото подобрене и визуализацията на работния процес.

Процесът от дефинирането на задачата до доставянето на продукта, е изложен на показ за участниците в него, а същевременно разработчиците на софтуера взимат работни задачи от така наречената „опашка“ (Backlog).

Използва система, при която се ограничава количеството на текуща работа във всеки един момент от работния процес, а допълнителни задачи могат да се поемат едва тогава, когато е възможно да има работен прогрес върху тях.

Фази:

- **Визуализиране на работния процес:** Работният процес се визуализира чрез използването на стена с колони и карти - Канбан дъска, на която с различни цветове карти

се изобразяват задачите. Колоните представляват различните състояния или стъпки на работния процес. Картите или задачите се преместват от един етап в друг.

- **Ограничаване на текущата работа:** Определяне на максимален брой задачи, които могат да бъдат изпълнявани едновременно. Текущата работа във всеки един етап от работния процес е лимитирана и нови задачи се „изтеглят“, когато има наличен капацитет в ресурса до достигане на лимита за все още незавършена работа.

- **Управление на потока:** Проследяване на задачите и минимизиране на пречките. Процесът на работа през всеки етап трябва да бъде следен, измерван и докладван. Чрез активно управление на процеса може да бъде оценено дали направените промени в системата ще имат положителни или отрицателни ефекти.

- **Уточняване на изискванията:** Определяне на ясни правила за преместване на задачите между колоните. Не е възможно да се обсъжда подобрието на даден процес, докато самия процес не е напълно изяснен. Без точно разбиране на целия работен процес, всяка дискусия може да бъде емоционална и субективна. Когато е налице коректно и точно разбиране на процеса и изискванията, е възможно да се премине към дискутиране на проблемите и предложения за подобряването му.

- **Използване на обратна връзка:** Редовно анализиране и подобряване на процеса. Организациите трябва да въведат обратна връзка за оценяване на влиянието на въведените промени.

Предимства:

- Гъвкавост и адаптивност.
- Визуализация на работния процес.
- Непрекъснато подобрене.

Недостатъци:

- Изисква постоянен мониторинг и управление.
- Може да бъде трудно за внедряване в големи екипи без ясна структура.

2.4. Спецификация на софтуерните изисквания

Спецификацията на софтуерните изисквания (Software Requirements Specification

- SRS) е документ, който подробно описва функционалните и нефункционалните изисквания

на софтуерен продукт [13]. Тя служи като ръководство за разработчиците, тестерите и останалите заинтересовани страни в процеса на разработка на софтуера.

SRS обикновено започва с въведение, което включва целта на документа и кои са целевите му читатели. След това обхватът на системата е описан, като се очертават границите на продукта. В общото описание на спецификацията се разглежда перспективата на продукта, като се обяснява как той се вписва в съществуващата система и среда. Основните функции на продукта са накратко представени, както и характеристиките на потребителите, които ще използват системата. Ограниченията, които трябва да бъдат взети под внимание, като технически или други ограничения, също са посочени.

Специфичните изисквания в SRS включват подробно описание на функциите, които системата трябва да изпълнява (функционални изисквания), както и изисквания за производителност, сигурност, поддръжка, използваемост и други нефункционални изисквания. Интерфейсните изисквания описват потребителски, хардуерни и софтуерни интерфейси.

Документът може да съдържа диаграми и модели като **диаграми на случаи на използване (Use Case Diagrams)**, които показват взаимодействията между потребителите и системата, **диаграми на активностите (Activity Diagrams)**, които показват потока на дейностите в системата, и **диаграми на класове (Class Diagrams)**, които показват структурните аспекти на системата.

Процесът на създаване на спецификация на софтуерните изисквания започва със събиране на изисквания чрез интервюта с потребители и заинтересовани страни, анкети и въпросници, наблюдение на текущи системи и процеси, както и провеждане на работни срещи и семинари. След събирането на изискванията те се анализират за пълнота, последователност и осъществимост. Изискванията се класифицират като функционални и нефункционални и се приоритизират. Документът се преглежда от всички заинтересовани страни, коригира се според обратната връзка и накрая се одобрява официално.

Спецификацията на софтуерните изисквания трябва да се поддържа и актуализира редовно в зависимост от промените в изискванията или в проекта. Добре изготвената SRS документация осигурява ясно разбиране на това, което трябва да бъде разработено, подобрява комуникацията между екипа за разработка и заинтересованите страни, подпомага планирането на проекта и разпределението на ресурсите и улеснява управлението на промените в изискванията. Създаването на добра спецификация на софтуерните изисквания е критично за успеха на софтуерния проект, тъй като тя служи като основа за целия процес на разработка, тестване и поддръжка на софтуера.

2.5. Техники за събиране на изисквания

Събирането на изисквания е критична стъпка в процеса на разработка на софтуер, която помага да се идентифицират нуждите и очакванията на потребителите и заинтересованите страни. Някои от основните начини за събиране на изисквания са [14]:

1. Интервюта – едни от най-разпространените методи за събиране на изисквания. Те могат да бъдат структурирани, полуструктурирани или неструктурирани. Структурираните интервюта следват предварително подготвен набор от въпроси, докато неструктурираните позволяват по-голяма гъвкавост и спонтанност.

2. Анкети и въпросници – ефективен начин за събиране на информация от голям брой хора. Те могат да бъдат използвани за получаване на количествени данни и често се създават с предварително дефинирани въпроси с фиксирани отговори.

3. Наблюдение на потребителя – то включва директно гледане на потребителите, докато те извършват своите обичайни дейности. Това може да помогне за разбиране на текущите процеси и идентифициране на проблеми или нужди, които потребителите може да не споменат сами.

4. Работни срещи и семинари – събират на едно място представители от различни групи заинтересовани страни, за да дискутират и дефинират изискванията. Те са особено полезни за съгласуване на различни гледни точки и постигане на консенсус.

5. Мозъчна Атака (Brainstorming) – техника за генериране на нови идеи и решения в групов формат. Всички участници са насърчавани да предложат колкото се може повече идеи, без критика или оценка по време на сесията. След това идеите се обсъждат и анализират.

6. Прототипиране – включва създаване на работещи модели или макети на системата, които могат да бъдат използвани за получаване на обратна връзка от потребителите. Това помага за визуализация на изискванията и идентифициране на потенциални проблеми на ранен етап.

7. Анализ на документи – анализът на съществуващи документи, като бизнес планове, технически спецификации, ръководства и отчетни документи, може да предостави ценна информация за изискванията. Това включва изучаване на съществуващите системи и процеси.

8. Use Case Analysis (Анализ на Случаи на Използване) – фокусира се върху идентифициране на взаимодействията между потребителите и системата. Случаите на използване описват конкретни сценарии, в които системата ще бъде използвана, което помага за разбиране на функционалните изисквания.

9. Фокус групи – те събират малка група от потребители или заинтересовани страни, които дискутират своите нужди и изисквания. Това предоставя възможност за интерактивна дискусия и събиране на разнообразни гледни точки.

10. Анализ на задачите (Task Analysis) – включва разбиване на дейностите на потребителите на по-малки задачи и изучаване на техните цели, стъпки и последователности. Това помага за разбиране на детайлите на работните процеси и изискванията, свързани с тях.

11. Игра на роли (Role-play) – някои системи трябва да отговарят на нуждите на различни типове потребители. Ролевата игра може да помогне да се гарантира, че нуждите на всички потребители са удовлетворени. По време на играта различни хора поемат ролите на различните типове потребители. Взаимодействието на различните роли помежду си помага да се изследват отделните системни изисквания от различни гледни точки и да се генерират нови идеи.

Тези техники могат да се използват самостоятелно или в комбинация, в зависимост от конкретните нужди и контекста на проекта. Ефективното събиране на изисквания е от съществено значение за успешната разработка на софтуер, като помага за осигуряване на продукта, който отговаря на очакванията и нуждите на потребителите.

2.6. Регулаторни изисквания

Регулаторните изисквания са критичен аспект, който гарантира съответствието на системата с правните и етични стандарти. Тези изисквания включват разнообразни области, като защита на личните данни, сигурност на информацията, стандарти за безопасност и качество на здравните услуги, както и съответствие с националните и международни регулации.

Един от основните аспекти на регулаторните изисквания е защитата на личните данни, особено в контекста на чувствителната медицинска информация. Регулации като Общия регламент за защита на данните (GDPR - General Data Protection Regulation) в Европейския съюз или Закона за защита на личните данни на здравеопазването (HIPAA - Health Insurance Portability and Accountability Act) в САЩ изискват стриктно спазване на принципите за поверителност и сигурност [15]. Това включва осигуряване на необходимите мерки за защита на данните от неоторизиран достъп, използване или разкриване, както и осигуряване на правата на пациентите върху техните лични данни, включително право на достъп, корекция и заличаване.

GDPR е въведен на 25 май 2018 г. и обхваща всички видове лични данни, включително здравни данни. Някои от основните му цели и принципи включват:

- 1. Справедливо и прозрачно обработване:** Данните трябва да се обработват по начин, който е законен, справедлив и прозрачен спрямо субекта на данните.
- 2. Ограничение на целите:** Личните данни могат да се събират само за конкретни, изрично посочени и легитимни цели и не могат да бъдат обработвани допълнително по начин, несъвместим с тези цели.
- 3. Минимизиране на данните:** Събирането на лични данни трябва да бъде ограничено до това, което е необходимо във връзка с целите, за които се обработват.
- 4. Точност:** Личните данни трябва да бъдат точни и при необходимост актуализирани.
- 5. Ограничение на съхранението:** Личните данни трябва да се съхраняват във форма, която позволява идентифициране на субектите на данните за период не по-дълъг от необходимия за целите, за които се обработват.
- 6. Цялостност и конфиденциалност:** Личните данни трябва да бъдат обработвани по начин, който гарантира подходяща сигурност на данните, включително защита срещу неразрешено или незаконно обработване и срещу случайна загуба, унищожаване или повреждане.

Една добре разработена система за онлайн консултации с медицински специалисти трябва да интегрира всички тези регулаторни изисквания в спецификацията на софтуерните изисквания. Това включва ясно дефиниране на всички правни и етични стандарти, които трябва да бъдат спазвани, както и осигуряване на механизми за мониторинг и оценка на съответствието с тези изисквания. По този начин се гарантира, че системата не само ще предоставя висококачествени медицински услуги, но и ще защитава правата и сигурността на пациентите.

2.7. Технологии за защита на данните

Защитата на данните е критичен аспект при създаването на система за онлайн срещи с медицински специалисти, като целта е да се гарантира сигурността и конфиденциалността на чувствителната медицинска информация [16]. Технологиите за защита на данни играят ключова роля в постигането на това и включват редица методи и инструменти.

Една от основните технологии за защита на данни е **криптирането**. Криптирането на данните осигурява, че информацията е нечетима за неоторизирани лица. Това се прилага както

за данните в покой (съхраняваните данни на сървърите), така и за данните в транзит (данните, които се предават между клиентите и сървърите). Използването на стандарти като **Advanced Encryption Standard (AES)** за криптиране на данните в покой и **Transport Layer Security (TLS)** за защитена комуникация в интернет е от съществено значение.

Съществуват два основни вида криптиране - симетрично и асиметрично.

Симетричното криптиране използва един и същ ключ за криптиране и декриптиране на данните. Този ключ трябва да бъде пазен в тайна и споделян само между страните, които трябва да имат достъп до криптираната информация. Симетричните алгоритми са известни с това, че са бързи и ефективни при обработка на големи обеми от данни, което ги прави подходящи за задачи като криптиране на файлове и дискове.

Основният недостатък на симетричното криптиране е управлението на ключовете. Тъй като един и същ ключ се използва и за криптиране, и за декриптиране, този ключ трябва да бъде сигурно предаван и съхраняван. Ако ключът бъде компрометиран, цялата криптирана информация е изложена на риск.

Асиметричното криптиране използва два различни, но математически свързани ключа: публичен и частен. Публичният ключ се използва за криптиране на данните и може да бъде свободно разпространяван, докато частният ключ се използва за декриптиране и трябва да се пази в тайна. Това решава проблема с управлението на ключовете при симетричното криптиране. Асиметричното криптиране е подходящо за задачи като сигурно предаване на ключове, цифрови подписи и установяване на сигурни канали за комуникация.

Основният недостатък на асиметричното криптиране е, че е по-бавно и изисква повече изчислителни ресурси в сравнение със симетричното криптиране. Това го прави по-малко подходящо за криптиране на големи обеми данни в реално време.

Автентикацията и авторизацията също са критични за защита на данните. Системата трябва да използва силни методи за автентикация, като **многофакторна автентикация (MFA - Multifactor authentication)**, която изисква от потребителите да представят повече от едно доказателство за самоличността си преди да получат достъп. Авторизацията, от своя страна, гарантира, че потребителите имат достъп само до ресурсите, които са им необходими за изпълнение на техните задачи, посредством роли и привилегии.

Защитните стени и антивирусните програми са допълнителни мерки за защита на данните. Те предотвратяват неоторизиран достъп до системата и защитават срещу зловреден софтуер. Защитните стени контролират входящия и изходящия трафик въз основа на предварително дефинирани правила за сигурност, докато антивирусните програми откриват и премахват зловреден софтуер.

Регулярните одити и тестове за сигурност са важни за поддържане на сигурността на системата. Периодичните одити помагат за идентифициране на уязвимости и несъответствия със стандартите за сигурност. Тестове като **Penetration Testing (Тест за проникване)** симулират кибератаки, за да оценят устойчивостта на системата срещу реални заплахи.

Логването и мониторингът на активностите в системата са ключови за откриване на и реагиране на инциденти със сигурността. Системата трябва да записва всички значими събития и действия, като например опити за вход, промени в данните и достъп до чувствителна информация. Тези логове трябва да бъдат редовно преглеждани и анализирани за аномалии, които биха могли да показват потенциални заплахи.

Бекъп и възстановяване на данни са важни технологии за защита на данните от загуба или повреждане. Редовното създаване на резервни копия на данните и съхранението им на сигурни места гарантира, че данните могат да бъдат възстановени в случай на инцидент. Трябва да бъдат въведени процедури за бързо възстановяване на данните, за да се минимизира времето на престой и въздействието върху потребителите.

Защитата на данни при създаването на система за онлайн срещи с медицински специалисти изисква комплексен подход, който комбинира множество технологии и методи. Криптирането, автентикацията и авторизацията, защитните стени, антивирусните програми, одитите, логването, мониторингът, и бекъпът са всички критични елементи, които допринасят за създаването на сигурна и надеждна система.

2.8. Потребителски интерфейс и използваемост:

Потребителският интерфейс (UI – User interface) и използваемостта са критични елементи при разработката на система за онлайн срещи с медицински специалисти. Добрият дизайн на интерфейса не само улеснява потребителите, но и подобрява общото преживяване, което е особено важно в контекста на здравеопазването. Ето основните принципи и аспекти на дизайн, които трябва да се вземат предвид [17]:

1. Простота и яснота

Интерфейсът трябва да бъде максимално опростен и интуитивен, за да могат потребителите лесно да се ориентират и използват системата. Това включва използване на ясно разпознаваеми икони, лесно четими шрифтове и минимализъм в дизайна. Основните функции и действия трябва да бъдат лесно достъпни и разбираеми дори за потребители с ограничени технически познания.

2. Консистентност

Консистентността в дизайна е ключова за създаването на предсказуем и лесен за използване интерфейс. Това включва използване на еднакви цветови схеми, шрифтове и стилове на бутони през цялата система. Менютата, диалозите и действията трябва да бъдат унифицирани, за да не объркват потребителите.

3. Достъпност

Системата трябва да бъде достъпна за всички потребители, включително тези с увреждания. Това включва поддръжка на технологии за достъпност като екранни четци, клавиатурна навигация и достатъчен контраст между текст и фон. Освен това, интерфейсът трябва да бъде адаптивен за използване на различни устройства, като компютри, таблети и смартфони.

4. Интуитивна Навигация

Навигацията трябва да бъде лесна и интуитивна, позволявайки на потребителите бързо да намерят и използват желаните функции. Това включва добре структурирани менюта, логически групирани опции и ясни пътища за връщане към предишни екрани. Полезни са и „breadcrumbs“ навигационни линкове, които показват пътя на потребителя през интерфейса.

5. Обратна Връзка

Потребителите трябва да получават незабавна и ясна обратна връзка за своите действия. Това включва визуални индикации за успешни операции, предупреждения за грешки и инструкции за коригиране на грешките. Например, след като потребителят резервира среща, системата трябва да показва потвърждение и подробности за срещата.

6. Персонализация

Възможността за персонализация на интерфейса може значително да подобри използваемостта. Потребителите трябва да могат да настройват предпочитанията си, като език, размер на шрифта и други визуални настройки. Освен това, системата може да предлага персонализирани препоръки въз основа на предишни взаимодействия и предпочитания.

7. Сигурност и Конфиденциалност

В контекста на онлайн срещи с медицински специалисти, сигурността и конфиденциалността са от изключителна важност. Интерфейсът трябва да предоставя ясна информация за мерките за защита на данните и да гарантира, че всички комуникации са криптирани. Потребителите трябва да се чувстват уверени, че личната им информация е защитена.

8. Лесен Достъп до Поддръжка

Системата трябва да предоставя лесен достъп до поддръжка и помощни ресурси. Това включва секции с често задавани въпроси (FAQ – Frequently Asked Questions), ръководства за потребители и опции за контакт с техническа поддръжка. Възможността за чат с поддръжка в реално време може също да бъде много полезна.

9. Потребителска контрол и свобода

Потребителите трябва да имат контрол над своите действия в системата и да могат лесно да отменят или променят направените действия. Това включва предоставяне на ясни опции за връщане назад, отмяна на операции и потвърждение на важни действия. Потребителите трябва да се чувстват уверени, че могат да коригират грешки и да променят решенията си.

10. Предотвратяване на грешки

Интерфейсът трябва да бъде проектиран така, че да предотвратява възможностите за грешки още преди те да се случат. Това може да включва използване на ограничения за въвеждане на данни, автоматично попълване на полета и валидиране на информацията в реално време. Когато грешка все пак се случи, системата трябва да предоставя ясни инструкции за коригиране.

Като следват тези принципи на дизайн, разработчиците могат да създадат система за онлайн срещи с медицински специалисти, която е лесна за използване, сигурна и ефективна. Това не само подобрява удовлетворението на потребителите, но и улеснява достъпа до качествени здравни услуги.

2.9. Методи за тестване на използваемост

Тестването на използваемост за система за онлайн срещи с медицински специалисти включва разнообразие от методи, които могат да бъдат разделени на качествени и количествени [18].

Качествени методи за тестване на използваемост

Качествените методи се фокусират върху разбирането на потребителското взаимодействие и впечатления чрез наблюдение и анализ на потребителското поведение в реална среда. Те предоставят дълбоко разбиране за проблеми и предизвикателства, които потребителите срещат при използването на системата. Някои от качествените методи включват:

- **Тестове с потребители:** Потребителите изпълняват задачи в системата, докато наблюдатели записват техните действия, коментари и проблеми. Този метод помага да се идентифицират трудности в навигацията, проблеми с интерфейса и общото удобство на употреба [18].

- **Контекстуални запитвания:** Потребителите се интервюират по време на използването на системата в тяхната естествена работна среда. Този подход помага да се разберат конкретни нужди и предпочитания, както и контекстът, в който се използва системата [19].

- **Картографиране на потребителското пътуване (Journey Mapping):** Анализ на стъпките, които потребителите извършват при използването на системата. Този метод идентифицира основните пътища и проблемни точки във взаимодействието на потребителите с интерфейса [20].

- **Дневници на потребителите (Diary Studies):** Потребителите записват своите впечатления, проблеми и успехи при използването на системата в рамките на определен период. Този метод предоставя детайлна информация за реалното използване и предизвикателствата, с които се сблъскват потребителите [21].

Количествени методи за тестване на използваемост

Количествените методи се фокусират върху събирането на числени данни и измерването на различни аспекти на използваемостта на системата. Те предоставят обективни показатели, които могат да се използват за сравнение и анализ на ефективността на интерфейса. Някои от количествените методи включват:

1. Анализ на метрики за използваемост: Събиране на данни за време за изпълнение на задачи, брой грешки, честота на използване на определени функции и степен на удовлетвореност на потребителите. Тези метрики предоставят количествени оценки за ефективността на интерфейса [22].

2. Онлайн анкети и въпросници: Потребителите попълват анкети след използване на системата, където споделят своите впечатления и мнения относно удобството на използване, функционалността и удовлетворението.

3. А/В тестове : Сравнение на две различни версии на интерфейса с цел определяне на по-ефективната от тях. Потребителите се разделят на две групи, като всяка от тях използва различна версия на системата, а резултатите се сравняват спрямо различни метрики [23].

Тези методи за тестване на използваемостта в комбинация предоставят комплексен подход за оценка и подобрене на системата за онлайн срещи с медицински специалисти, като съчетават дълбоко разбиране за потребителските нужди с обективни числени оценки за ефективността и удобството на интерфейса.

2.10. Интеграция и взаимодействие с други системи

Интеграцията и взаимодействието с други системи, както и интероперабилността, са ключови аспекти при разработването на система за онлайн срещи с медицински специалисти. Те гарантират, че системата може ефективно да обменя информация и да работи съвместно с други медицински и административни системи.

Интеграцията с други системи е критична за осигуряване на безпроблемен обмен на данни и информация между различни здравни платформи и услуги. Това включва следните ключови компоненти:

- **Електронни здравни досиета (EHR – Electronic health record):** Интеграцията с EHR системи позволява на лекарите да имат достъп до пълната медицинска история на пациента по време на онлайн консултация. Това улеснява по-доброто диагностициране и лечение, като предоставя важна информация за алергии, предишни заболявания и медикаменти [24].

- **Системи за управление на болници (HMS):** Свързването със системите за управление на болници улеснява администрирането на пациентските данни, планирането на срещи и управлението на медицинските ресурси. Това подобрява координацията между различните отдели и улеснява работата на медицинския персонал [25].

- **Аптечни системи:** Интеграцията с аптечни системи позволява на лекарите да изписват електронни рецепти директно по време на онлайн консултацията. Пациентите могат бързо и лесно да получат предписаните медикаменти от избраната аптека [26].

- **Системи за лабораторни тестове:** Възможността за директно свързване с лабораторни системи позволява на лекарите да заявяват и получават резултатите от лабораторни изследвания по електронен път. Това ускорява процеса на диагностициране и лечение [27].

- **Телемедицински платформи:** Интеграцията с други телемедицински платформи позволява на системата да предлага по-широк обхват от услуги и специалисти. Това осигурява по-добро обслужване на пациентите, като им предоставя достъп до различни експерти и специализирани услуги [26].

Интероперабилността е способността на системата да обменя и използва информация с други системи по стандартен и съвместим начин. Това е особено важно в здравеопазването, където обменът на точна и навременна информация е критичен. Основните аспекти на интероперабилността включват [28]:

- **Стандарти за данни:** Използването на общоприети стандарти за данни като HL7, FHIR и DICOM [29] улеснява обмена на информация между различни здравни системи. Тези стандарти осигуряват съвместимост и консистентност на медицинските данни.

HL7 (Health Level Seven) е набор от международни стандарти за обмен, интеграция, споделяне и извличане на електронна здравна информация. Той се използва широко в здравеопазването за осигуряване на съвместимост между различни информационни системи. HL7 обхваща различни аспекти на здравната информация, включително клинични документи, административни данни и лабораторни резултати.

FHIR (Fast Healthcare Interoperability Resources) е стандарт за обмен на здравна информация, разработен от HL7. Той е създаден с цел да улесни и модернизира обмена на здравна информация чрез използване на съвременни уеб технологии. FHIR комбинира най-доброто от предишните стандарти на HL7 и добавя нови възможности за гъвкавост и лесна интеграция.

DICOM (Digital Imaging and Communications in Medicine) е международен стандарт за съхранение, предаване и визуализация на медицински изображения и свързана информация. Той се използва широко в радиологията, кардиологията и други медицински области, където медицинските изображения играят важна роля.

- **Сигурност и конфиденциалност:** Гарантирането на сигурността и конфиденциалността на данните е критично при обмена на медицинска информация. Това включва използването на криптиране, автентикация и контрол на достъпа, за да се защитят данните на пациентите.

- **API интеграции:** Използването на Приложно-програмен интерфейс (**API - Application Programming Interfaces**) позволява лесна и ефективна интеграция между различни системи. API осигуряват стандартни методи за обмен на данни и функционалности между системите [30].

API осигурява абстракция на сложността, като скрива детайлите за вътрешната реализация на функциите и услугите. Това позволява на разработчиците да използват функционалностите на API без да разбират детайлите на тяхната реализация.

API осигурява стандартизиран начин за взаимодействие, което гарантира съвместимост и улеснява интеграцията между различни системи. Стандарти като REST и SOAP са често използвани за уеб API.

API често включват механизми за сигурност, като автентикация и авторизация, за да се гарантира, че само оторизирани потребители и приложения имат достъп до ресурсите и данните.

REST (Representational State Transfer) е архитектурен стил за проектиране на мрежови приложения [30]. RESTful API използват HTTP протокола и се основават на ресурси, които се идентифицират чрез **URL (Uniform Resource Locator)** адреси. Те поддържат операции като GET, POST, PUT и DELETE за манипулиране на ресурсите. RESTful API са популярни поради своята простота, лекота на използване и мащабируемост.

SOAP (Simple Object Access Protocol) е протокол за обмен на структурирана информация в разпределена среда [30]. SOAP използва **XML (Extensible Markup Language)** за съобщенията и поддържа различни транспортни протоколи като **HTTP (Hypertext Transfer Protocol)** и други. SOAP API осигуряват висока степен на сигурност и надеждност, но са по-сложни и тежки в сравнение с RESTful API.

- **Съвместимост с мобилни и уеб платформи:** Системата трябва да бъде съвместима с различни устройства и платформи, включително мобилни телефони, таблети и настолни компютри. Това осигурява лесен достъп до системата за различни потребители, независимо от техните предпочитания и устройства.

- **Автоматизация на процесите:** Интероперабилността позволява автоматизация на рутинни процеси като записване на срещи, актуализиране на пациентски данни и генериране на доклади. Това намалява административното натоварване и подобрява ефективността на здравните услуги.

2.11. Изводи от литературния обзор

Литературният обзор показва, че системите за онлайн консултации с медицински специалисти имат значителен потенциал за подобряване на достъпа до медицински услуги и повишаване на ефективността на здравеопазването. Въпреки това, разработката на такива системи изисква прецизно дефиниране на софтуерните изисквания, включително както функционални, така и нефункционални изисквания.

Въпреки значителния напредък в областта на телемедицината, съществуват редица области, които изискват допълнителни изследвания. Това включва разработка на по-сигурни и

конфиденциални методи за защита на данните, подобряване на използваемостта на системите и осигуряване на по-добра интеграция с други здравни информационни системи.

Бъдещите изследвания могат да се фокусират и върху използването на нови технологии като изкуствен интелект и машинно обучение за подобряване на функционалността и ефективността на системите за онлайн консултации.

3. Експериментална част

3.1. Обекти на изследване

3.1.1. Цел на проекта

Целта на този документ е да предостави подробно описание на функционалните и нефункционалните изисквания на системата за управление на онлайн консултации с лекар. Документът служи като ръководство за екипа за разработка, заинтересованите страни и потребителите на системата. Той очертава целите и задачите на проекта, функционалните и нефункционалните изисквания на системата и очакваните резултати. Основните групи, които поддържа системата са: лекари, пациенти и разработчици (администратор).

Разработчиците трябва да се консултират с този документ и неговите редакции като единствен източник на изисквания за проекта. Те не трябва да считат никакви изявления за изисквания, писмени или устни, за валидни, докато не се появят в този документ или неговата редакция.

3.1.2. Предвидена аудитория

Предвидената аудитория за този документ включва заинтересовани страни по проекта, членове на екипа за разработка и крайни потребители на системата за онлайн консултации с лекар. Заинтересованите страни по проекта включват доставчици на здравни услуги, пациенти и администратори, които участват в разработването и внедряването на системата.

Членовете на екипа за разработка включват разработчици на софтуер, тестери, ръководители на проекти и системни архитекти, които отговарят за проектирането, изграждането и тестването на системата.

Крайните потребители на системата са доставчици на здравни услуги и пациенти, които ще използват системата, за да управляват своите консултации.

3.1.3. Обхват на проекта

Обхватът на проекта за системата за онлайн консултации с медицински специалисти включва проектиране, разработване и внедряване на уеб-базирано приложение, което

позволява на пациентите да планират срещи онлайн, а на доставчиците на здравни услуги да управляват ефективно графика си. Системата също така ще предоставя актуализации в реално време на пациентите и лекарите относно статуса на срещите, намалявайки шансовете за пропуснати или двойно резервирани срещи. Обхватът на проекта включва и тестване, обучение и внедряване на системата, за да се гарантира, че тя отговаря на нуждите на заинтересованите страни и потребителите.

3.1.4. Обзор на документа

Първо, документът определя целта на проекта, която е разработването на система за онлайн консултации с медицински специалисти.

Второ, предвидената аудитория включва разработчиците на софтуер, проектните мениджъри, медицинските специалисти и заинтересованите страни като здравни организации и институции.

Трето, обхватът на проекта обхваща функционалностите и ограниченията на системата, включително взаимодействието между пациенти и медицински специалисти чрез видеоконференции, чат и обмен на документи.

Четвърто, документът съдържа гледната точка на продукта, неговите характеристики, ограничения в дизайна и изпълнението, предположения и зависимости, както и съществуващите проблеми, които системата решава.

Пето, разделът със системни изисквания представя както функционалните, така и нефункционалните изисквания.

Шесто, моделите за анализ включват диаграма на случаите на използване, диаграма на класовете, ER диаграма, диаграма на потока на данни и диаграми на последователността.

Седмо, архитектурата на системата разглежда системните хардуерни и софтуерни изисквания.

Осмо, документът описва приемните тестови случаи, които ще се използват за верификация на системата.

Девето, накрая е включен примерен потребителски интерфейс на системата, който служи като ръководство за разработчиците.

3.1.5. Гледна точка на продукта

Системата е предназначена да осигури интуитивен потребителски интерфейс, който е лесен за използване както от пациенти, така и от лекари. Системата ще позволи на пациентите да насрочват срещи онлайн, да преглеждат медицинската си история и да комуникират с лекари. Системата също така ще предостави на лекарите лесен за използване интерфейс за управление на срещи с пациенти, преглед на медицински истории и комуникация с пациенти.

Системата също така ще предостави платформа за разработчиците да персонализират приложението, за да отговори на специфичните нужди на медицинския персонал. Системата ще бъде проектирана да бъде гъвкава и мащабируема, способна да се адаптира към променящите се нужди на медицинските практики.

3.1.6. Характеристики на продукта

Системата за управление на консултацията с лекар е проектирана да предоставя набор от функции за лекари, пациенти и разработчици. Следните са основните функции на системата:
За пациенти:

- **Онлайн планиране на срещи:** Пациентите могат да планират срещи със своите лекари онлайн чрез системата.
- **Преглед на медицинската история:** Пациентите могат да преглеждат медицинската си история през системата, включително минали срещи и медицински досиета.
- **Сигурни съобщения:** Пациентите могат да комуникират със своите лекари сигурно чрез системата.
- **Заявки за рецепта:** Пациентите могат да поискат повторно попълване на рецепта чрез системата.
- **Напомняне за срещи:** Пациентите могат да получават автоматизирани напомняния за предстоящи срещи.

За лекари:

- **Управление на срещите:** Лекарите могат да управляват срещите на пациентите чрез системата, включително планиране, пренасрочване и отмяна на срещи.
- **Управление на медицински досиета:** Лекарите могат да преглеждат и управляват медицински досиета на пациенти чрез системата.

- **Сигурни съобщения:** Лекарите могат да комуникират с пациенти сигурно чрез системата.
- **Управление на рецепти:** Лекарите могат да управляват рецепти на пациенти чрез системата.
- **Напомняне за час:** Лекарите могат да изпращат автоматизирани напомняния за час на пациентите.

За разработчици(admin):

- **Персонализиране:** Разработчиците могат да персонализират системата, за да отговарят на специфичните нужди на медицинските практики. Това включва модифициране на потребителския интерфейс, добавяне на нови функции и интегриране с други системи.
- **Интеграция:** Разработчиците могат да интегрират системата със съществуващите системи за електронни здравни досиета. Това ще позволи на системата да комуникира безпроблемно с други системи, намалявайки административната тежест и подобрявайки ефективността на медицинските лица. Системата трябва да бъде проектирана така, че да се интегрира лесно с други системи, като се използват стандартни за индустрията протоколи.
- **Сигурност:** Разработчиците могат да гарантират, че системата е проектирана с мисъл за сигурността и поверителността на данните. Това включва прилагане на подходящи контроли за достъп, криптиране на данни и процедури за архивиране на данни. Системата трябва да бъде проектирана така, че да отговаря на всички съответни разпоредби, за да се гарантира, че информацията за пациента е защитена по всяко време.
- **Тестване:** Разработчиците могат да извършат стриктно тестване на системата, за да гарантират, че е надеждна и без грешки. Това включва тестване на системата при различни условия и сценарии, за да се гарантира, че тя може да обработва големи обеми трафик и да не губи производителност. Системата трябва да бъде тествана както за функционалност, така и за сигурност, като се използват както автоматизирани, така и ръчни техники за тестване.
- **Поддръжка:** Разработчиците могат да осигурят текуща поддръжка на системата. Това включва наблюдение на системата за грешки и проблеми, извършване на редовни актуализации и предоставяне на техническа помощ на потребителите. Системата трябва да бъде проектирана така, че да бъде лесна за поддръжка, като се използват стандартни за индустрията инструменти и техники.

3.1.7. Ограничения на дизайна и изпълнението

- Системата трябва да бъде разработена с помощта на уеб-базирана платформа, която може да бъде достъпна отвсякъде и по всяко време.
- Системата трябва да отговаря на всички стандартни разпоредби и закони, свързани с медицинската практика и поверителността на данните.
- Системата трябва да има удобен за потребителя интерфейс, който е лесен за навигация и разбираем от всички потребители.
- Системата трябва да бъде мащабируема и да може да обработва голям брой потребители и срещи.
- Системата трябва да има надеждна система за архивиране и възстановяване, за да се гарантира, че данните няма да бъдат загубени поради непредвидени обстоятелства.

3.1.8. Предположения и зависимости

- Системата предполага, че всички лекари са се съгласили да участват в системата и да предоставят своята възможност за записване на час онлайн.
- Системата предполага, че пациентите имат достъп до компютър или мобилно устройство с интернет връзка, за да използват системата.
- Системата зависи от наличието на интернет и хардуера, използван за достъп до нея.
- Системата зависи от точността и пълнотата на информацията, предоставена от лекарите и пациентите по време на регистрация и записване на час.
- Системата предполага, че лекарите и пациентите ще запазят своите идентификационни данни за вход поверителни, за да предотвратят неоторизиран достъп.

3.1.9. Съществуващи проблеми, които системата решава

- Дълго време на чакане за пациентите да видят лекар поради неефективна система за насрочване на срещи.
- Трудност за пациентите да намерят и резервират час при лекари, които отговарят на специфичните им изисквания.
- Неефективно използване на времето на лекаря поради пропуснати часове или отмяна в последния момент.

- Трудност за лекарите да управляват своите срещи и наличност ръчно, което води до свръхзаписване или двойно записване на часове.
- Невъзможност за проследяване и управление на досиетата на пациентите и срещите ефективно, което води до грешки и забавяне на лечението.

Чрез справянето с тези проблеми, системата за управление на консултациите с лекари ще осигури по-ефективен начин за пациентите и лекарите да управляват срещите си, да подобрят грижите за пациентите и да рационализират медицинската практика.

3.1.10. Системни изисквания

3.1.10.1. Функционални изисквания

1. Системата трябва да позволява на пациентите и лекарите да влизат (login):

- Те трябва да въведат своето потребителско име и парола.
- Предоставената информация трябва да е валидна.
- Достъпът се предоставя / отказва.

2. Системата трябва да позволява регистрация на пациенти и лекари:

- В случай на пациент събира потребителска информация (имена, дата на раждане, адрес, телефон, имейл, парола и др.).
- В случай на лекар събира докторска информация (имена, имейл, парола, пол, дата на раждане, отдел, адрес, телефон и др.).
- Проверява дали информацията е валидна:
 - * Паролата не е празна.
 - * Паролата и потвърждението за паролата са еднакви.
 - * Имейл адресът не е използван преди.
 - * При валидна информация, запазва и добавя потребителят към базата данни.

3. Системата трябва да позволява на пациентите и лекарите да излизат (logout):

- Излизане от системата, когато потребителят щракне върху бутона за излизане.

4. Системата трябва да позволява на пациентите да записват час:

- Системата трябва да провери дали пациентът е влязъл или не.

- Пациентът избира отделението и болницата, които го интересуват.
- Системата трябва да показва свободен час за консултация при конкретния лекар.
- Системата генерира уникална справка за резервация за всяка среща.
- Системата ще изпрати имейл за потвърждение, когато срещата е направена.

5. Системата трябва да позволява на пациентите да правят запитвания:

- Системата ще изисква от клиента да предостави своя имейл, за да може да бъде изпратен отговор.
- Системата трябва да изисква въвеждане на коментар, описващ каквито и проблеми клиентът да има.

6. Системата трябва да позволява на пациентите да търсят налични лекари от определено медицинско направление (област):

- Пациентът трябва да може да въведе собственото или фамилното име на лекаря, както и да може да търси лекари от дадена медицинска област (направление).
- Системата показва всички лекари, които отговарят на критериите на пациента.
- Системата трябва да показва свободните часове на лекаря и възможност за запазване.

7. Системата трябва да позволява на пациентите да променят или анулират своите резервации:

- Системата трябва да позволява резервациите да бъдат променяни, без да се налага повторно въвеждане на цялата информация за пациента.
- Пациентът трябва само да предостави своя референтен номер за резервация.
- Системата ще направи необходимите актуализации след направените промени.

8. Системата трябва да позволява на пациентите и лекарите да преглеждат и променят своя профил:

- Пациентите и лекарите въвеждат новата информация.
- След това тази информация замества старата информация в базата данни.

9. Системата трябва да позволява на лекарите да определят своето налично време:

- Лекарят ще въведе времето, в което ще бъде на разположение.
- Тази информация се записва в базата данни.

10. Системата трябва да позволява на администратора да влезе:

- Потребителят въвежда своето потребителско име и парола.
- Предоставената информация трябва да е валидна.
- Достъпът се предоставя/отказва.

11. Системата трябва да позволява на администратора да управлява потребителите:

- Системата позволява на администратора да има достъп до базата данни и да добавя нови потребители.
- Системата позволява на администратора да изтрие всеки потребител поради определени правила от базата данни.
- Системата позволява на администратора да променя информацията за пациента или лекаря като фамилно име, имейл, парола, отдел и др.

12. Системата трябва да позволява на администратора да изтрива минали консултации от системата:

- След като мине определено време от консултацията, администраторът трябва да може да я изтрие от базата данни.

13. Системата трябва да позволява на администратора да управлява и има достъп до базата данни:

- Системата позволява на администраторът да имат достъп до базата данни и да управлява информацията от нея.

3.1.10.2. Нефункционални изисквания

1. Надеждност:

- Системата трябва да бъде налична, когато е поискана услуга от потребителите: Системата трябва винаги да е готова и работеща, така че когато потребителят иска да я използва, да е налична.
- Системата трябва да има много нисък процент на отказ: той трябва да бъде възможно най-малък, за предпочитане под 0,01%.

2. Производителност:

- Системата трябва да има добро време за реакция.

- Времето за зареждане на потребителския интерфейс трябва да отнеме по-малко от две секунди.
- Информацията за влизане трябва да бъде потвърдена в рамките на пет секунди.
- Заявките трябва да връщат резултати в рамките на пет секунди.
- Системата трябва да може да постигне много за определено време.
- Системата трябва да може да издържи на голямо натоварване.
- Системата трябва да може да издържа множество хора едновременно.
- Системата трябва да работи без грешки, докато работи с огромен набор от данни.
- Системата трябва да бъде прецизна и точна при работа с данни.
- Степента на грешки в системата трябва да бъде минимална.

3. Сигурност:

- Всички външни комуникации между сървъра за данни на системата и клиентите трябва да бъдат криптирани.
- За да се гарантира, че системата е защитена, достъпът до различните подсистеми ще бъде защитен от потребителски екран за влизане, изискващ потребителско име и парола.
- Разрешенията за достъп до системни данни могат да се променят само от системния администратор на данни: Системният администратор трябва да е единственият, който има право да разрешава достъп до системните данни.
- Всички системни данни трябва да се архивират на всеки 24 часа и резервните копия да се съхраняват на сигурно място, което не е в същата сграда като системата: Това се прави, за да се избегне загуба на информация в случай на срыв на системата. Системните данни трябва да се съхраняват в устройство за съхранение, напр. твърд диск, CD, флаш устройство или може да се съхранява във файлове.

4. Използваемост:

- Системата трябва да включва добре структурирани ръководства за потребителя.
- Системата трябва да има информативни съобщения за грешка.
- Системата трябва да обяснява какво е направил погрешно потребителят.
- Системата трябва да показва къде точно може да бъде открита грешката.
- Системата трябва да обясни как да се поправи грешката.
- Съобщението за грешка трябва да е лесно за разбиране.
- Системата трябва да има добре оформен графичен потребителски интерфейс.

- Системата трябва да е удобна за потребителя.
- Системата трябва да е лесна за научаване както за начинаещи, така и за потребители с опит в подобни системи.
- Системата трябва да е ефективна за всички потребители.
- Потребителят трябва да разбира какво прави системата.
- Потребителят трябва да се чувства доволен от работата със системата.

5. Безопасност:

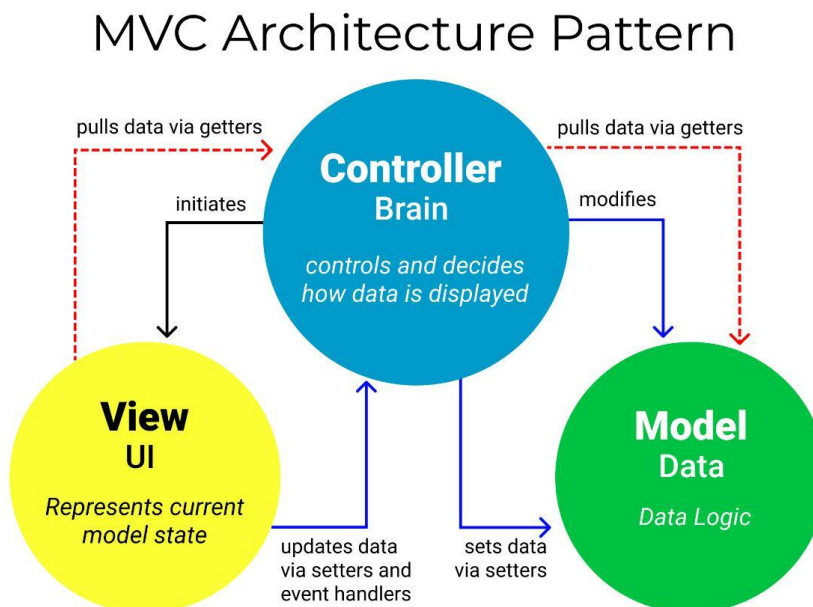
- Системата трябва да поддържа добро архивиране: Поддържането на резервни копия гарантира, че базата данни на системата е защитена, което означава, че в случай на авария или инцидент системата може лесно да бъде възстановена.

6. Възможност за поддръжка (Supportability):

- Системата трябва да може да се прехвърля от една среда в друга.
- Трябва да работи на Microsoft Windows, Linux и macOS.
- Системата трябва да е лесна за поддръжка.
- Поддръжката на системата трябва да бъде рентабилна.
- Системата трябва лесно да се адаптира към направените промени.
- Системата трябва да може да се справя с допълнителни международни конвенции като езици, часови зони, стилове.
- Часовата зона трябва да съответства на тази на потребителя.
- Системата трябва да функционира правилно на различни платформи като хардуер, браузър, виртуална машина и т.н.

3.1.11. Архитектура на системата

3.1.11.1. Архитектура Model-View-Controller (MVC)



Фиг. 8 Общ модел на MVC архитектура [39]

Системата за онлайн среща с лекари използва архитектура **Model-View-Controller (MVC)**. Тя се състои от три основни компонента: модел (Model), изглед (View) и контролер (Controller) [39].

Моделът представлява данните и бизнес логиката на приложението. Той е отговорен за съхраняването, обработката и манипулирането на данните. За системата за консултации с медицински специалисти представлява обекти като пациенти, лекари, часове и евентуално фактуриране.

Изгледът е отговорен за представянето на данните на потребителя. Това може да бъде HTML страница, графичен интерфейс или друг тип изглед. За системата за консултации с медицински специалисти представя потребителски интерфейси за пациенти, които резервират часове, лекари, които виждат графици, и администратори, които управляват системата.

Контролерът свързва модела и изгледа. Той получава заявките от потребителя, обработва ги, извлича необходимата информация от модела и я предоставя на изгледа за показване. Контролерът също така приема действия от потребителя (обработва заявки като

резервиране на час, извличане на данни за часове, актуализиране на графици) и ги обработва, като актуализира модела спрямо тези действия.

MVC архитектурата помага за разделянето на отговорностите между компонентите на приложението и го прави по-лесно за поддръжка и разширение. Тя също така позволява на различни разработчици да работят по различни части от приложението без да нарушават функционалността на другите компоненти. Архитектурата също така насърчава повторната употреба на кода, което улеснява разработването на системата.

3.1.11.2. Системни хардуерни изисквания

Процесор: Двухъдрен 2 GHz или по-висок.

Памет: Поне 4 GB RAM.

Съхранение: минимум 20 GB място на твърдия диск.

Мрежа: Gigabit Ethernet адаптер за LAN свързаност.

Дисплей: Минимум 1920 x 1080 резолюция.

Входни устройства: Мишка и клавиатура.

3.1.11.3. Системни софтуерни изисквания:

Операционна система: Windows, Mac OS или Linux със съвместима версия на Java Runtime Environment (JRE).

Java Development Kit (JDK): Версия 8 или по-нова.

Сървър за приложения (Application Server): Apache Tomcat или GlassFish.

Уеб браузър: Google Chrome, Mozilla Firefox, Microsoft Edge или Safari.

Система за управление на бази данни: MySQL Community Edition версия 5.7 или по-нова.

Среда за разработка: Интегрирана среда за разработка (IDE) като Eclipse или IntelliJ IDEA.

Framework: Spring Framework версия 5 или по-нова, Hibernate ORM версия 5 или по-нова и Apache Struts версия 2 или по-нова.

Софтуер за криптиране: OpenSSL или подобен софтуер за криптиране за сигурно предаване на данни по интернет.

Антивирусна и защитна стена: Антивирусен софтуер за защита срещу зловреден софтуер и защитна стена за защита срещу неоторизиран достъп.

Софтуер за архивиране и възстановяване: Софтуер за архивиране и възстановяване, за да се гарантира наличността на системата в случай на загуба на данни или повреда на системата.

Изискванията за системен хардуер и системен софтуер бяха избрани въз основа на технологичния стек на системата за управление на лекарски прегледи, която използва Java и MySQL. Тези изисквания са необходими за правилното функциониране на системата, тъй като гарантират, че хардуерните и софтуерните компоненти отговарят на необходимите спецификации, за да може системата да работи гладко и ефективно.

Хардуерните изисквания като процесор, памет, съхранение, мрежа, дисплей и входни устройства бяха избрани въз основа на нуждите на системата, като например количеството обработени данни, броя на едновременните потребители и резолюцията на дисплея, необходима за потребителския интерфейс.

Софтуерните изисквания, включително операционната система, Java Development Kit (JDK), сървър за приложения, уеб браузър, система за управление на бази данни, инструменти за разработка, рамки, софтуер за криптиране, антивирусна програма и защитна стена, както и софтуер за архивиране и възстановяване, бяха избрани въз основа на технологиите и инструменти, необходими за разработване, внедряване и поддръжка на системата.

Спазването на тези хардуерни и софтуерни изисквания е от решаващо значение за успеха на системата, тъй като те гарантират, че системата е стабилна, сигурна и надеждна и че може да бъде разработена, тествана и внедрена ефективно.

3.2. Методи за изследване

3.2.1. Езикът за програмиране Java

Java е утвърден език за програмиране с множество предимства, които го правят подходящ за разработката на сложни системи, включително система за онлайн консултации с медицински специалисти. Ще бъдат разгледани предимствата на Java в контекста на този проект и ще бъде направена съпоставка с други популярни езици за програмиране като Python, C# и JavaScript.

Предимства на Java:

1. Платформена независимост – Java е известна със своят принцип "пиши веднъж, изпълнявай навсякъде" (write once, run anywhere) [31]. Това се постига чрез **Java Virtual**

Machine (JVM), която позволява на Java приложенията да работят на всяка платформа, която поддържа JVM. Това е особено важно за система за онлайн консултации, която може да се използва на различни устройства и операционни системи.

2. Сигурност – Java има вградени функции за сигурност, като управление на паметта и автоматично почистване на ненужни обекти (**garbage collection**) [32], които намаляват вероятността от уязвимости като препълване на буфера (**buffer overflow**) [32]. Това е критично за системи, които обработват чувствителна медицинска информация.

3. Поддръжка и библиотечна екосистема – Java разполага с богата библиотечна екосистема, включително много библиотеки и фреймуърци за разработка на уеб приложения като **Spring Framework** и **Hibernate ORM (Object Relational Mapping)** [33]. Те предлагат готови решения за често срещани задачи като управление на бази данни, обработка на заявки и сигурност.

4. Мащабируемост – Java е известна със своята способност за изграждане на мащабируеми и надеждни backend системи. Това е важно за онлайн платформи, които могат да имат голям брой потребители и трябва да осигурят стабилно представяне дори при високо натоварване.

5. Многонишковост (Multithreading) – Java има вградена поддръжка за многонишковост, което позволява ефективното използване на процесорните ресурси и подобрява производителността на системата [34]. Това е особено полезно за реално-времеви приложения като видеоконферентни връзки.

Съпоставка с други езици за програмиране и извод:

Python е известен със своята простота и четимост на кода [35]. Въпреки това, той може да бъде по-бавен от Java поради интерпретирането му в реално време. Python е подходящ за прототипиране и бързо разработване, но може да не е толкова ефективен за мащабируеми и високопроизводителни системи.

C# е език, разработен от Microsoft, и е подобен на Java по отношение на синтаксис и функционалности [35]. Той е много мощен за разработка на приложения за Windows. Въпреки това, Java има предимството на платформена независимост, докато C# е тясно свързан с .NET екосистемата и Windows.

JavaScript е основният език за уеб разработка, използван в клиентската част на уеб приложенията [35]. Node.js позволява използването на JavaScript и за сървърна страна. Въпреки че е много добър за разработка на уеб базирани приложения, JavaScript може да не е толкова ефективен като Java за изграждане на сложни и мащабируеми бекенд системи.

Java е най-подходящият език за програмиране за изграждане на система за онлайн консултации с медицински специалисти поради своите предимства като платформена независимост, висока сигурност, богата библиотечна екосистема, мащабируемост и поддръжка за многонишковост. Въпреки че други езици като Python, C# и JavaScript също имат своите предимства, Java предоставя най-балансираното и надеждно решение за разработка на такъв тип система.

3.2.2. Избор на система за управление на бази данни

Изборът на подходяща система за бази данни е критично важен за успешното функциониране на система за онлайн консултации с медицински специалисти. Видовете системи за бази данни биват [36]:

Релационни бази данни (RDBMS - Relational database management system):

Релационните бази данни използват структурирани таблици за съхранение на данни, където всяка таблица съдържа редове и колони. Основни характеристики включват:

- Структуриран формат: Данните са строго структурирани и организирани в таблици с предварително дефинирани схеми.
- **SQL (Structured Query Language)**: Използват се SQL за управление и манипулиране на данните.
- Цялостност на данните: Поддържат се ограничения като първични ключове, чужди ключове и уникални ограничения, които осигуряват цялостта и консистентността на данните.

Нерелационни бази данни (NoSQL):

Нерелационните бази данни са по-гъвкави по отношение на структурата на данните и могат да съхраняват неструктурирани или полу-структурирани данни. Основни характеристики включват:

- Гъвкавост на схемата: Данните могат да бъдат съхранявани без предварително дефинирани схеми.
- Разнообразие от модели на данни: Включват документо-ориентирани бази данни, **key-value stores**, графови бази данни и **Column Family Store (CFS)**.
- Висока скалируемост: NoSQL базите данни са проектирани за лесно мащабиране хоризонтално (**horizontal scaling**).

Съпоставка между релационни и нерелационни бази данни

1. Предимства на релационните бази данни:

- Структурирани данни: Подходящи за приложения, където данните са добре структурирани и има ясни отношения между тях.
- **ACID Properties:** Гарантират атомичност, последователност, изолация и издръжливост на транзакциите, което е критично за финансови и медицински приложения.
- Сложни заявки: SQL позволява изпълнението на сложни заявки за извличане и манипулиране на данните.

2. Предимства на нерелационните бази данни:

- Гъвкавост: Подходящи за динамично променящи се данни и непредвидими схеми.
- Мащабируемост: Лесно мащабируеми хоризонтално, което е важно за големи приложения с висока натовареност.
- Бърз достъп до данни: Оптимизирани за бързи операции по четене и писане.

Медицинските данни изискват висока степен на точност и консистентност. Релационните бази данни, с техните **ACID свойства (Atomicity, Consistency, Isolation, and Durability)**, гарантират, че данните остават консистентни и точни дори при сложни транзакции и многократни едновременни достъпи [37].

Медицинските записи често имат сложни релации между различни видове данни (напр. пациенти, лекари, диагнози, лечения). SQL предоставя мощни инструменти за изпълнение на сложни заявки, които са необходими за извличане на свързаните данни.

Релационните бази данни предлагат изчерпателни механизми за управление на достъпа до данните, което е критично за защита на конфиденциалната медицинска информация.

3. MySQL

MySQL е една от най-утвърдените и широко използвани релационни бази данни в света [38]. Тя е доказала своята стабилност и надеждност в множество големи и сложни проекти.

MySQL има голяма и активна общност, която предлага богат набор от ресурси, включително документация, уроци и форуми за поддръжка. Това улеснява разработчиците при решаването на възникнали проблеми.

MySQL предлага висока производителност и ефективност, особено при четене и писане на големи обеми данни. Това е важно за система за онлайн консултации, която трябва да обработва множество заявки в реално време.

MySQL предлага надеждни механизми за управление на достъпа и защита на данните, включително **SSL/TLS криптиране**, ролево базирано управление на достъпа и поддръжка на външни системи за автентикация.

MySQL предлага множество инструменти за управление, като **MySQL Workbench**, което улеснява администрацията на базата данни, създаването на диаграми, изготвянето на заявки и други административни задачи.

Въпреки че както релационните, така и нерелационните бази данни имат своите предимства, релационните бази данни са по-подходящи за изграждане на система за онлайн консултации с медицински специалисти поради тяхната способност да осигурят цялостност и сигурност на данните, както и възможността за изпълнение на сложни заявки. MySQL, като утвърдена и надеждна релационна база данни, е най-добрият избор за такъв проект благодарение на своя богат набор от ресурси, производителност, мащабируемост и поддръжка от голяма общност.

3.2.3. Анализ на изискванията чрез диаграми

Unified Modeling Language (UML) е стандартен език за моделиране на софтуерни системи, който предоставя начин за визуализация, спецификация, конструиране и документиране на артефакти на софтуерните системи [40]. UML използва различни видове диаграми за представяне на различни аспекти на софтуерния дизайн и архитектура. В контекста на изграждане на система за онлайн консултации с медицински специалисти, UML диаграмите играят ключова роля в разбиране, планиране и комуникация на изискванията и архитектурата на системата. Видовете диаграми в UML биват:

1. Диаграма на случаите на използване (Use case diagram) – Диаграмата на случаите на използване е един от най-основните и важни елементи на UML [41]. Тя описва взаимодействието между потребителите (актьори) и системата чрез различни случаи на използване (use cases).

Основната ѝ цел е да идентифицира и документира основните функционалности на системата от гледна точка на потребителите. Състои се от актьори (пациенти, лекари, администратори), случаи на използване (регистрация, резервация на час, онлайн консултация), асоциации между актьорите и случаите на използване.

В системата за онлайн консултации, диаграмата може да включва актьори като пациенти и лекари, и случаи на използване като „регистрация на пациент“, „резервация на час за консултация“, „онлайн видео консултация“.

2. Диаграма на класовете (Class Diagram) – Диаграмата на класовете показва статичната структура на системата, като дефинира класовете, техните атрибути, методи и връзките между тях [42]. Примерни компоненти са класове (Patient, Doctor, Appointment), атрибути (name, date), методи (schedule(), cancel()) и връзки (асоциации, наследяване).

Класовете са основните градивни елементи в класовата диаграма. Те представляват обекти или концепции в системата и дефинират структурата и поведението на тези обекти. Класът е като шаблон за създаване на обекти и съдържа атрибути и методи.

Атрибутите са данни, които са свързани с обекта от класа. Те описват характеристиките на класа и могат да бъдат различни типове, като цяло число, текст, дата и други. Например, класът „Пациент“ може да има атрибути като име, имейл и дата на раждане.

Методите са функции, които определят действията, които могат да се извършват върху обектите от класа. Те дефинират как класът ще реагира на различни събития или заявки. Например, класът „Пациент“ може да има методи като регистриране, вход и запазване на консултация.

Връзките между класовете показват как те са свързани и взаимодействат един с друг. Има няколко основни типа връзки:

Асоцииране (Association) - Показва връзка между два класа, които взаимодействат помежду си. Например, пациент може да има множество консултации.

Наследяване (Inheritance) - Показва, че един клас (подклас) наследява атрибутите и методите на друг клас (базов клас). Например, класовете „Пациент“ и „Медицински специалист“ могат да наследяват общи атрибути от клас „Потребител“.

Агрегация (Aggregation) - Показва такава връзка, където един клас (цялото) съдържа обекти от друг клас (частта), но частите могат да съществуват самостоятелно. Например, класът „Консултация“ има агрегационна връзка с класа „Препоръки“. Това означава, че една консултация може да съдържа множество препоръки, но препоръките могат да съществуват самостоятелно от консултацията. Ако консултацията бъде унищожена, препоръките могат да продължат да съществуват или да бъдат свързани с друга консултация.

Композиция (Composition) - Специален вид агрегация, при която частите не могат да съществуват самостоятелно от цялото. Например, консултация не може да съществува без пациент.

3. ER Diagram (Entity–relationship model) – Това е модел за визуално представяне на структурите на база данни, който показва как данните са организирани и как различните елементи на данните са свързани помежду си. ER диаграмите се използват основно в процеса на проектиране на бази данни, за да се идентифицират и описват отношенията между различните субекти (entities) в системата [43].

Основни компоненти на ER диаграмата са субектите (entities), които са основни обекти или концепции в системата, които трябва да бъдат представени в базата данни. Субектите се представят с правоъгълници (Пациент, Лекар, Консултация).

Друг вид компоненти са атрибутите (attributes), които са свойства или характеристики на субектите. Атрибутите се представят с елипси и са свързани с правоъгълника на субекта (Име, Имейл, Дата на раждане).

Съществуват и така наречените връзки (relationships), които показват как субектите са свързани един с друг. Връзките се представят с ромбове, които са свързани с правоъгълниците на субектите. Има няколко вида връзки:

- **Едно към едно (1:1):** Един субект от тип А може да бъде свързан с точно един субект от тип Б и обратно.
- **Едно към много (1:M):** Един субект от тип А може да бъде свързан с много субекти от тип Б.
- **Много към много (M:M):** Много субекти от тип А могат да бъдат свързани с много субекти от тип Б.

4. Диаграмата на потока от данни (Data Flow Diagram) – Това е визуален модел, който илюстрира как данните се движат в система. Той се използва за анализиране и проектиране на информационни системи, като подчертава процесите, данните, файловете и потоците между тях [44].

Основни компоненти на диаграмата на потока от данни:

- **Процесите (Processes)** представляват дейности или функции, които обработват входящи данни и генерират изходящи данни. Процесите се изобразяват често като кръгове или овални фигури със зададени номера.

- **Данните (Data)** представляват информацията, която се обработва и движи в системата. Данните се показват често като стрелки, които свързват процесите, файловете или външните субекти.

- **Хранилищата за данни (Data Stores)** са запамятаващите средства или хранилища, където данни могат да бъдат запазени за по-късна употреба. Те се изобразяват често като правоъгълници.

- **Външните субекти (External Entities)** са външни системи, които комуникират с анализираната система и доставят или получават данни. Те се показват често като правоъгълници със стрелки, които показват посоката на потока на данни.

5. Диаграма на последователност (Sequence Diagram) – Тя се използва за визуално представяне на последователността на взаимодействия между обекти в определен сценарий. Тази диаграма показва как обектите взаимодействат помежду си чрез изпращане на съобщения в определен ред [45]. Диаграмите на последователност са особено полезни за моделиране на динамични аспекти на системата и за разбиране на потока на контрол и данни.

Основни елементи на диаграмата на последователност:

- **Актьори (Actors)**, които представляват външни участници в системата, като потребители или външни системи. Те се изобразяват като човекоподобни икони или правоъгълници.

- **Обектите (Objects)** са компоненти на системата, които участват във взаимодействието. Те се представят като правоъгълници с имената на обектите и класа.

- **Жизнените линии (Lifelines)** са вертикални прекъснати линии, които показват съществуването на обектите през времето.

- **Съобщенията (Messages)** представляват хоризонтални стрелки между жизнените линии, които показват изпращането на съобщения (методи) от един обект към друг. Съобщенията могат да бъдат синхронни (с очакване на отговор) или асинхронни (без очакване на отговор).

- **Активациите (Activation Bars)** са вертикални правоъгълници на жизнените линии, които показват времето, през което обектът е активен или изпълнява даден процес.

3.2.4. Методи за верификация и валидация

А) Приемните тестови случаи (Acceptance Test Cases) са критична част от процеса на тестване на софтуер, особено за системи като онлайн консултации с медицински специалисти, където сигурността, точността и ефективността са от съществено значение. Тези тестове се извършват, за да се гарантира, че системата отговаря на предварително определените изисквания и че работи според очакванията на потребителите и заинтересованите страни [46].

Стъпки за създаване на приемни тестови случаи:

1. Разбиране на изискванията - Първата стъпка в създаването на приемни тестови случаи е детайлното разбиране на изискванията на системата. Това включва функционалните и нефункционалните изисквания, описани в спецификацията на софтуерните изисквания (SRS).

2. Дефиниране на критериите за приемане - Критериите за приемане са конкретни условия, които системата трябва да изпълни, за да бъде приета от клиента или крайните потребители. Тези критерии са базирани на изискванията и могат да включват функционални, производителни и сигурностни аспекти.

3. Разработване на приемни тестови случаи - След като критериите за приемане са дефинирани, следващата стъпка е разработването на приемните тестови случаи. Тези тестове трябва да обхващат всички аспекти на системата и да проверяват дали тя отговаря на критериите за приемане.

4. Изпълнение на тестовете - Изпълнението на приемните тестови случаи се извършва след като системата е разработена и готова за тестване. Това включва провеждане на тестовете в реална или симулирана среда, за да се провери дали системата отговаря на критериите за приемане.

5. Анализ и отчет на резултатите - След като тестовете са изпълнени, следващата стъпка е анализирането на резултатите и съставянето на отчет за тях. Този отчет трябва да включва обобщение на тестовете, откритите проблеми и препоръки за коригиране на отклоненията.

Б) Спецификацията на случаите на употреба (Use Case Specification) е документ, който подробно описва как потребителите (актьори) взаимодействат със система, за да постигнат определени цели [47]. Тази спецификация служи като ръководство за разработка, тестване и поддръжка на системата и включва информация за различни сценарии, които показват как системата трябва да се държи в определени ситуации.

Компоненти на спецификацията на случаите на употреба:

1. Име на случая на употреба - кратко и ясно име, което описва целта на случая на употреба.
2. Идентификатор - уникален идентификатор за всеки случай на употреба, който улеснява проследяването и референцирането.
3. Описание - кратко описание на целта и контекста на случая на употреба.

4. Актьори - списък на актьорите, които участват в случая на употреба (напр. потребители, системи, външни участници).
5. Предусловия - условията, които трябва да бъдат изпълнени, за да може случаят на употреба да започне.
6. Основен поток (Main Flow) - подробно описание на основните стъпки, които актьорите и системата предприемат за изпълнение на случая на употреба.
7. Алтернативни потоци (Alternate Flows) - описание на различни пътища, които могат да бъдат следвани, ако възникнат определени условия или грешки по време на основния поток.
8. Следусловия - описва състоянието на системата след успешното изпълнение на случая на употреба.
9. Изключения (Exception Flows) - Описание на действията, които трябва да бъдат предприети при възникване на грешки или изключения.
10. Зависимости - връзки и зависимости между този и други случаи на употреба.

4. Обсъждане на експерименталните резултати, графики и таблици

4.1. Подробна спецификация на случаите на употреба

REQ1. Системата трябва да позволява на пациентите и лекарите да влизат.

REQ2. Системата трябва да позволява регистрация на пациенти и лекари.

REQ3. Системата трябва да позволява на пациенти и лекари да излизат.

REQ4. Системата трябва да позволява на пациентите да записват час.

REQ5. Системата трябва да позволява на пациентите да правят запитвания.

REQ6. Системата трябва да позволява на пациентите да търсят свободни лекари.

REQ7. Системата трябва да позволява на пациентите да променят или отменят своя час.

REQ8. Системата трябва да позволява на пациентите и лекарите да преглеждат своя профил.

REQ9. Системата трябва да позволява на лекарите да определят своето налично време.

REQ10. Системата трябва да позволява на администратора да влезе.

REQ11. Системата трябва да позволява на администратора да управлява потребителите

REQ12. Системата трябва да позволява на администратора да изтрива минали срещи.

REQ13. Системата трябва да позволява на администратора да има достъп до базата данни.

Таблица 1 Спецификация на случаите на употреба за вход (login) в системата

Use Case UC-1:		Вход (Login)
Съответни изисквания:		REQ1, REQ3, REQ4, REQ7, REQ8, REQ9, REQ10, REQ11, REQ12, REQ13
Инициращ актьор:		Пациент, лекар, администратор
Цел на актьора:		Да влезе в своя потребителски профил
Предпоставки:		Системата показва менюто за въвеждане на потребителско име и парола.
Постусловие (Postcondition):		Системата трябва да пренасочи потребителя към началната страница.
Поток от събития за основен сценарий за успех:		
	1.	Потребителят отваря системата и щраква върху „Вход“.
	2.	Системата показва меню за въвеждане на потребителско име и парола.
	3.	Потребителят въвежда информацията и щраква върху бутона за изпращане.
	4.	Системата показва „Успешно влизане“
Поток от събития за разширения (алтернативни сценарии):		
Потребителят въвежда невалидно потребителско име или парола		
	1.	Системата показва предупредително съобщение, че въведените потребителско име или парола са грешни и изисква повторно въвеждане на информация.

REQ2. Системата трябва да позволява регистрация на пациенти и лекари.

REQ4. Системата трябва да позволява на пациентите да записват час.

REQ7. Системата трябва да позволява на пациентите да променят или отменят своя час.

REQ8. Системата трябва да позволява на пациентите и лекарите да преглеждат своя профил.

REQ9. Системата трябва да позволява на лекарите да определят своето налично време.

Таблица 2 Спецификация на случаите на употреба за регистрация в системата

Use Case UC-2:		Регистрация
Съответни изисквания:		REQ2, REQ4, REQ7, REQ8, REQ9
Инициращ актьор:		Пациент, доктор
Цел на актьора:		Да се създаде акаунт, за да се резервира час за пациента и лекарят да може да потвърди и управлява този час.
Предпоставки:		Системата показва менюто за лекар или пациент за въвеждане на лична информация.
Постусловие (Postcondition):		Системата трябва да пренасочи потребителя към страницата за вход.
Поток от събития за основен сценарий за успех:		
	1.	Потребителят отваря системата и натиска върху „Регистрация“, за да създаде акаунт
	2.	Системата показва на потребителя страница, за да получи основна информация за потребителя като име, фамилия, потребителско име, дата на раждане, парола и др.
	3.	Потребителят въвежда информацията и щраква върху ОК, за да бъде създаден акаунт
	4.	Системата показва „Успешна регистрация“
Поток от събития за разширения (алтернативни сценарии): Потребителят въвежда невалидно потребителско име или парола		
	1.	Системата показва предупредително съобщение, че въведените потребителско име или парола са невалидни или вече в употреба и изисква повторно въвеждане на информация.

REQ1. Системата трябва да позволява на пациентите и лекарите да влизат.

REQ2. Системата трябва да позволява регистрация на пациенти и лекари.

REQ4. Системата трябва да позволява на пациентите да записват час.

REQ7. Системата трябва да позволява на пациентите да променят или отменят своя час.

REQ13. Системата трябва да позволява на администратора да има достъп до базата данни.

Таблица 3 Спецификация на случаите на употреба за резервация на час за преглед

Use Case UC-3		Резервация на час за преглед
Съответни изисквания:		REQ1, REQ2, REQ4, REQ7, REQ13
Инициращ актьор:		Пациент
Цел на актьора:		Да направи онлайн резервация на час за консултация с лекар
Предпоставки:		Потребителят трябва да създаде акаунт и да влезе
Постусловие (Postcondition):		Системата трябва да показва графика на лекаря
Поток от събития за основен сценарий за успех:		
	1.	Пациентът прави заявка за резервация, като кликне върху бутона за записване на час
	2.	Системата показва на пациента меню за избор на отдел и място.
	3.	Пациентът въвежда информация.
	4.	Системата показва списък с лекари, които отговарят на критериите.
	5.	Пациентът избира лекар.
	6.	Системата показва графикът на избрания лекар.
	7.	Пациентът избира предпочитания час и ден.
	8.	Системата показва „Успешно резервиран час“.
Поток от събития за разширения (алтернативни сценарии):		
Потребителят иска да отмени резервацията		
	1.	Потребителят натиска върху бутона „Отказ от резервация“.
	2.	Системата показва съобщение, че анулирането е извършено

REQ1. Системата трябва да позволява на пациентите и лекарите да влизат.

REQ2. Системата трябва да позволява регистрация на пациенти и лекари.

REQ8. Системата трябва да позволява на пациентите и лекарите да прегледват своя профил.

Таблица 4 Спецификация на случаите на употреба за преглед и промяна на данните

Use Case UC-4:		Преглед и промяна на данните в профила.
Съответни изисквания:		REQ1, REQ2, REQ8
Инициращ актьор:		Пациент, Доктор
Цел на актьора:		Да промени личната си информация.
Предпоставки:		Потребителят трябва да влезе в менюто за редактиране на профила и да промени своите данни
Постусловие (Postcondition):		Системата трябва да покаже съобщение за потвърждение, за да се гарантира актуализиране на информацията
Поток от събития за основен сценарий за успех:		
	1.	Потребителят влиза в своя профил (account page)
	2.	Системата показва страницата на личния акаунт на потребителя.
	3.	Потребителят избира опция за редактиране на информация
	4	Системата показва поле за писане на нови данни на потребителя.
	4.	Потребителят въвежда нова информация и запазва направените промени, като щракне върху бутона „Запиши“
	5.	Системата показва „Данните са актуализирани“.

REQ1. Системата трябва да позволява на пациентите и лекарите да влизат.

REQ2. Системата трябва да позволява регистрация на пациенти и лекари.

REQ9. Системата трябва да позволява на лекарите да определят своето налично време.

Таблица 5 Спецификация на случаите на употреба за задаване на свободно време

Use Case UC-5	Задаване на свободно време	
Съответни изисквания:	REQ1, REQ2, REQ9	
Инициращ актьор:	Доктор	
Цел на актьора:	Да зададе наличен времеви интервал в графика	
Предпоставки:	Потребителят трябва да създаде акаунт и да влезе	
Постусловие (Postcondition):	Системата трябва да показва новия график на лекаря	
Поток от събития за основен сценарий за успех:		
	1.	Докторът натиска върху бутона за профил.
	2.	Системата показва страницата на акаунта (account page).
	3.	Докторът избира свободно време в показания график.
	4.	Системата показва новия наличен времеви интервал в графика.

REQ1. Системата трябва да позволява на пациентите и лекарите да влизат.

REQ2. Системата трябва да позволява регистрация на пациенти и лекари.

REQ4. Системата трябва да позволява на пациентите да записват час.

REQ7. Системата трябва да позволява на пациентите да променят или отменят своя час.

Таблица 6 Спецификация на случаите на употреба за отмяна на записан час

Use Case UC-6		Отмяна на записан час
Съответни изисквания:		REQ1, REQ2, REQ4, REQ7
Инициращ актьор:		Пациент, Доктор
Цел на актьора:		Да отмени предварително запазен час
Предпоставки:		В системата трябва да има предварително записан час
Постусловие (Postcondition):		Системата трябва да покаже съобщение „Срещата е отменена“
Поток от събития за основен сценарий за успех:		
	1.	Потребителят натиска върху „Отмяна на среща“ в календара.
	2.	Системата показва поле за потвърждение „сигурни ли сте, че искате да отмените срещата“.
	3.	Потребителят избира „ДА“.
	4.	Системата изтрива часът и показва съобщение „Срещата с лекар е отменена“.
Поток от събития за разширения (алтернативни сценарии):		
Потребителят решава да не анулира срещата		
	1.	Потребителят избира НЕ в полето за потвърждение.
	2.	Системата показва началния график за срещи без промени.

REQ6. Системата трябва да позволява на пациентите да търсят свободни лекари.

Таблица 7 Спецификация на случаите на употреба за търсене на лекар

Use Case UC-7		Търсене на лекар
Съответни изисквания:		REQ6
Инициращ актьор:		Пациент
Цел на актьора:		За търсене на определен лекар
Предпоставки:		Потребителят трябва да създаде акаунт и да влезе
Постусловие (Postcondition):		Системата трябва да показва информация за лекаря
Поток от събития за основен сценарий за успех:		
	1.	Потребителят натиска бутона за търсене.
	2.	Системата показва формата за търсене.
	3.	Потребителят въвежда данни на лекаря и натиска „Търсене“.
	4.	Системата показва информация за лекаря.
Поток от събития за разширения (алтернативни сценарии): Докторът не е в системата.		
	1.	Системата показва „Лекарят не съществува“

REQ1. Системата трябва да позволява на пациентите и лекарите да влизат.

REQ10. Системата трябва да позволява на администратора да влезе.

REQ11. Системата трябва да позволява на администратора да управлява потребителите

Таблица 8 Спецификация на случаите на употреба за управление на потребителите

Use Case UC-8:	Управление на потребителите	
Съответни изисквания:	REQ1, REQ10, REQ11	
Инициращ актьор:	Администратор	
Цел на актьора:	Да добави потребител или да го изтрие от системата	
Предпоставки:	Потребителят трябва да влезе в системата само чрез администраторски права	
Постусловие (Postcondition):	Системата трябва да покаже съобщение, за да потвърди, че актуализацията е завършена	
Поток от събития за основен сценарий за успех:		
	1.	Потребителят натиска върху страницата Лекар/Пациент.
	2.	Системата показва списък с Лекар/Пациент.
	3.	Потребителят избира опция за добавяне/изтриване на лекар/пациент
	4.	Системата показва формуляр с полета за попълване.
	4.	Потребителят въвежда информация и щраква върху бутона за добавяне / Потребителят избира опция за изтриване на профил.
	5.	Системата показва съобщение, което показва, че потребителят е добавен или изтрит.

REQ10. Системата трябва да позволява на администратора да влезе.

REQ13. Системата трябва да позволява на администратора да има достъп до базата данни.

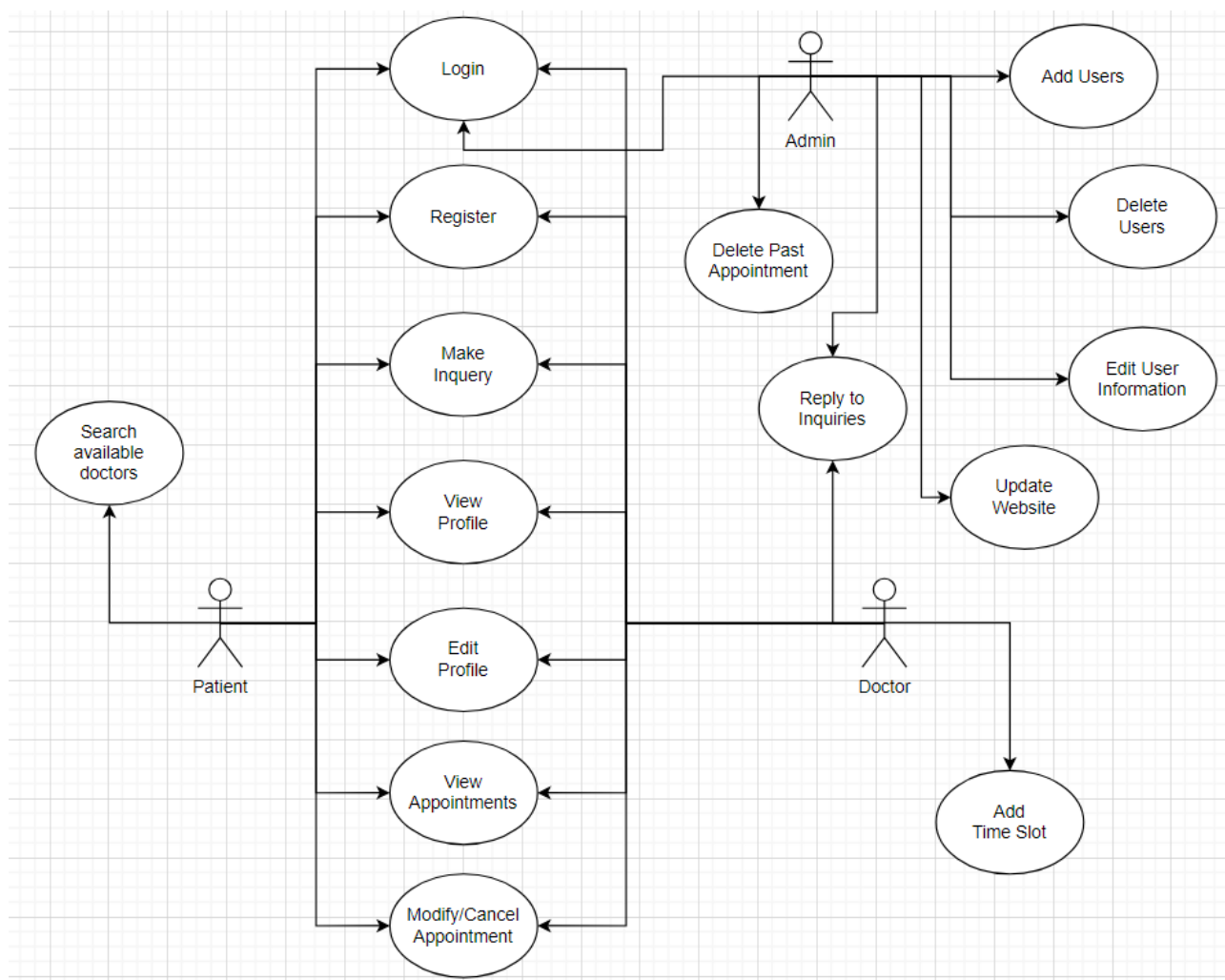
Таблица 9 Спецификация на случаите на употреба за управление и достъп до базата

Use Case UC-9:	Управление и достъп до база данни	
Съответни изисквания:	REQ10, REQ13	
Инициращ актьор:	Администратор на база данни	
Цел на актьора:	Да управлява, добавя, редактира, актуализира или изтрива данни от базата	
Предпоставки:	Потребителят трябва да влезе в системата само чрез администраторски права	
Постусловие (Postcondition):	Системата трябва да покаже съобщение, че модифицираните данни се съхраняват в базата данни	
Поток от събития за основен сценарий за успех:		
	1.	Потребителят натиска върху бутона за управление на базата данни
	2.	Системата показва цялата информация в базата данни
	3.	Потребителят може да добавя, премахва или актуализира конкретна част от базата данни
	4.	Потребителят запазва промените, като натисне върху бутона за запазване
	5.	Системата показва съобщение, за да информира потребителя, че промените са направени успешно

Таблица 9. Спецификация на случаите на употреба за управление и достъп до базата

4.2. Модели за анализ

4.2.1. Диаграма на случаите на използване (Use case diagram)



Фиг. 9 Модел на диаграма на случаите на използване за системата

На Фиг. 9 е представената диаграма на случаите на използване, която илюстрира различните взаимодействия между различните участници (потребители) и системните функции в онлайн система за медицински консултации. Диаграмата очертава основните случаи на употреба за три типа потребители: пациенти, лекари и администратори.

1. Актьори:

Пациент - потребител, който търси медицински консултации.

Лекар - медицински специалист, който предоставя консултации.

Администратор - системен администратор, отговорен за управлението на системата и потребителските акаунти.

2. Случаи на употреба:

За пациенти:

- Login: Позволява да се влиза в системата.
- Register: Позволява да се създават нови акаунти в системата.
- Make Inquiry: Позволява да се правят запитвания в системата.
- View Profile: Позволява да се преглежда информацията в потребителския профил.
- Edit Profile: Позволява да се редактират подробностите на потребителския профил.
- View Appointments: Позволява да се преглеждат планираните срещи.
- Modify/Cancel Appointment: Позволява да се променят или отменят съществуващи срещи.
- Search Available Doctors: Позволява на пациентите да търсят налични лекари за консултация.

За лекари:

- Имат достъп до всички функционалности на пациентите без Search Available Doctors
- Reply to Inquiries: Позволява на лекарите да отговарят на запитвания на пациенти.
- Add Time Slot: Позволява на лекарите да добавят налични времеви интервали за консултации.

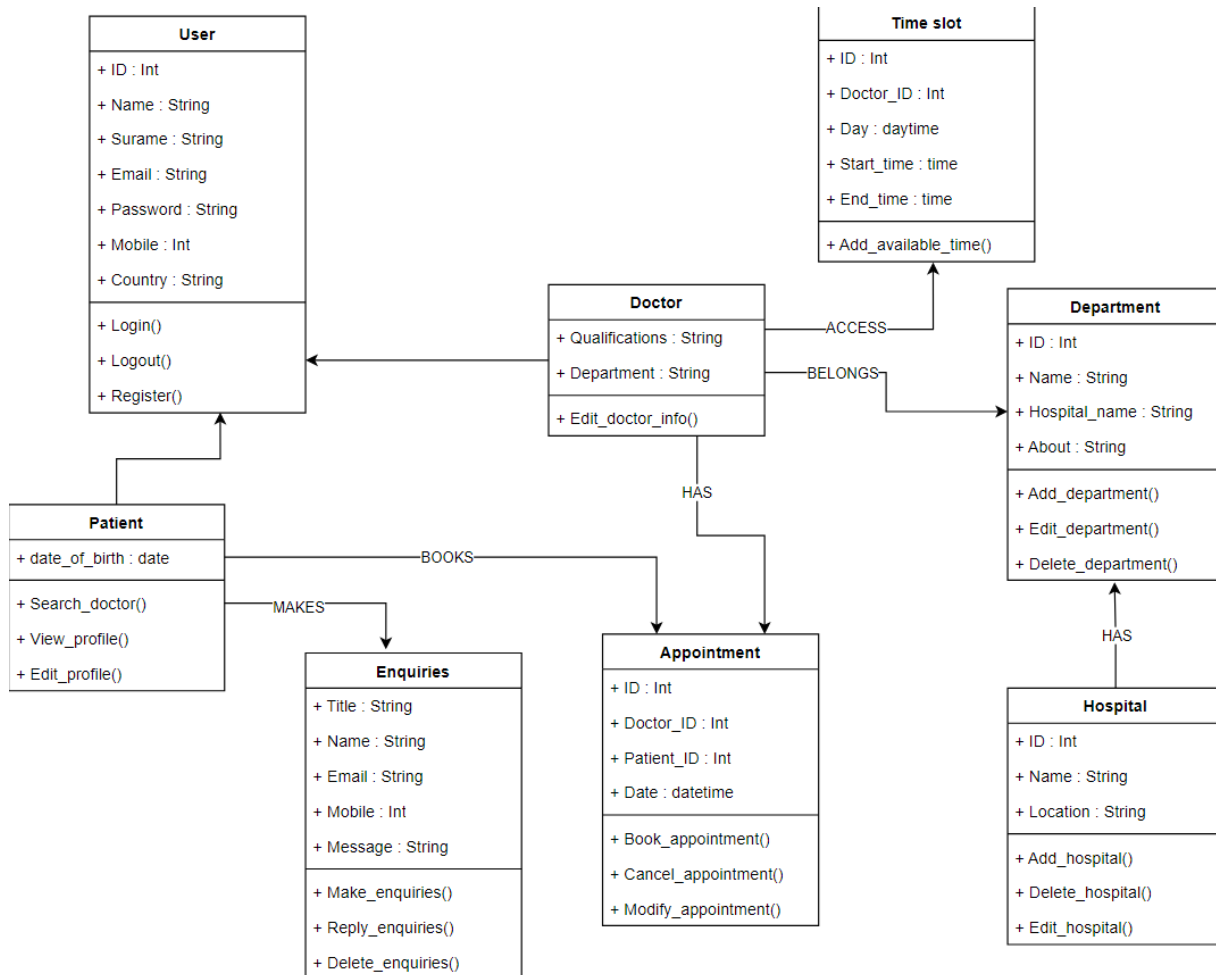
За администратори:

- Login: Позволява на администраторите да влизат в системата.
- Add Users: Позволява на администраторите да добавят нови потребители.
- Delete Users: Позволява на администраторите да изтриват съществуващи потребителски акаунти.
- Edit User Information: Позволява на администраторите да редактират потребителски данни.
- Update Website: Позволява да се актуализира информацията на уебсайта.
- Delete Past Appointments: Позволява да се изтриват срещи, които вече са преминали.
- Reply to Inquiries: Позволява на администраторите да отговарят на общи запитвания.

Диаграмата на случаите на използване предоставя преглед на взаимодействията в системата за онлайн медицински консултации. Пациентите могат да управляват своите профили и срещи, докато търсят и взаимодействат с лекари. Лекарите могат да управляват своята наличност и да отговарят на запитвания на пациенти. Администраторите наблюдават цялостното управление на системата, като гарантират безпроблемната работа на платформата,

като управляват потребителите и поддържат цялостта на данните и функционалността на системата.

4.2.2. Диаграма на класовете (Class Diagram)



Фиг. 10 Модел на диаграма на класовете за системата

Класовата диаграма на Фиг. 10 представя структурата на система за назначаване на медицински прегледи, като подробно описва различните класове, техните атрибути, методи и връзки.

1. Класове и атрибути:

- User (Потребител):

Атрибути:

ID : Int

Name : String

Surname : String

Email : String

Password : password

Mobile : Int

Country : String

Методи: Login() ; Logout() ; Register()

- Patient (Пациент):

Атрибути:

date of birth : date

Методи: Search_doctor() ; View_profile() ; Edit_profile()

- Enquiries (Запитвания):

Атрибути:

Title : String

Name : String

Email : String

Mobile : Int

Message : String

Методи: Make_enquiries() ; Reply_enquiries() ; Delete_enquiries()

- Doctor (Лекар):

Атрибути:

Qualifications : String

Department : String

Методи: Edit_doctor_info()

- Appointment (Среща):

Атрибути:

ID : Int

Doctor_id : Int

Patient_id : Int

Date: datetime

Методи: Book_appointment() ; Cancel_appointment() ; Modify_appointment()

- Time Slot (Времеви интервал):

Атрибути:

ID : Int

Doctor_ID : Int

Day : daytime

Start_time : time

End_time : time

Методи: Add_available_time()

- Department (Отдел):

Атрибути:

ID : Int

Name : String

Hospital_name : String

About : String

Методи: Add_department() ; Edit_department() ; Delete_department()

- Hospital (Болница):

Атрибути:

ID : Int

Name : String

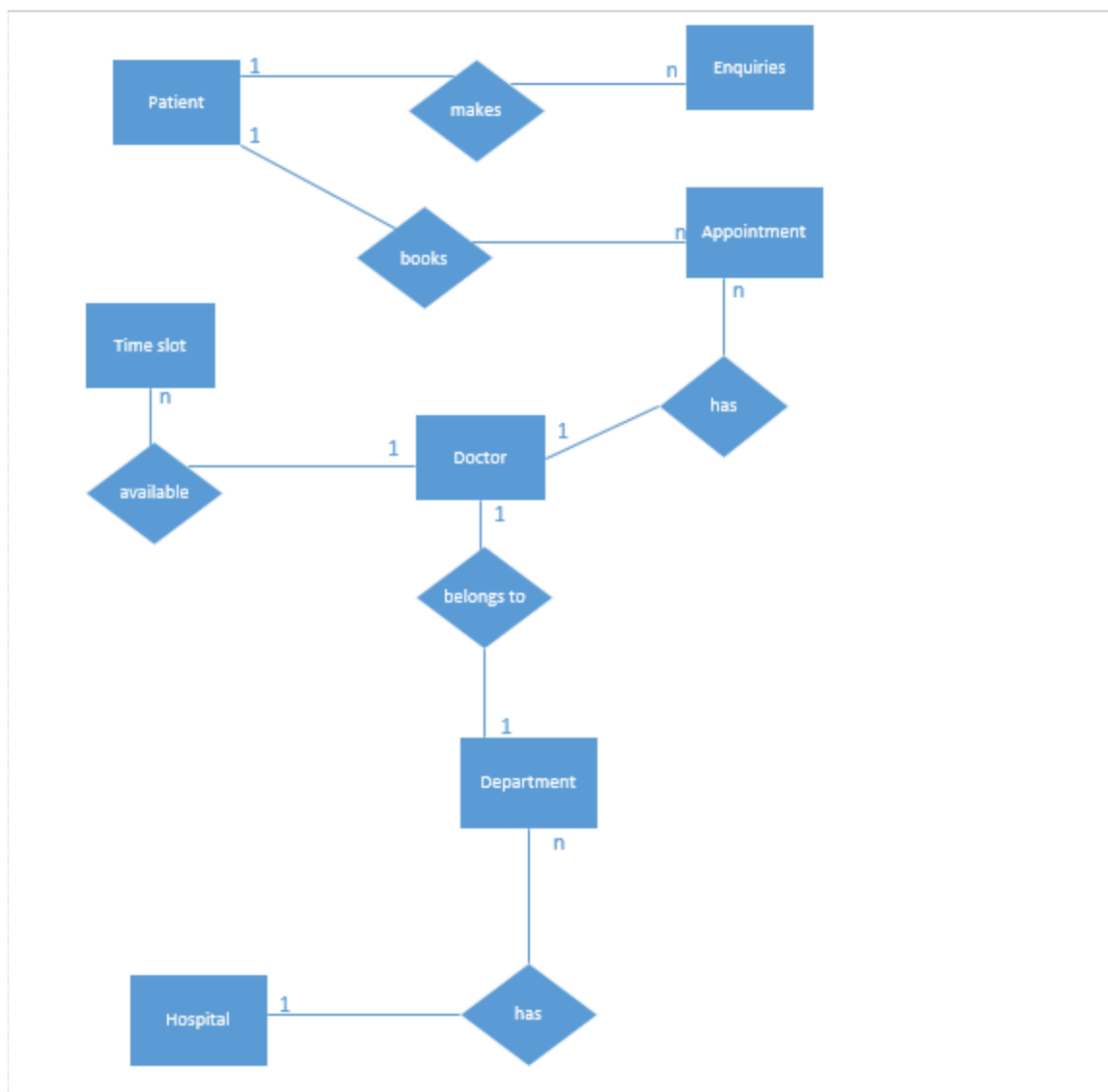
Location : String

Методи: Add_hospital() ; Delete_hospital() ; Edit_hospital()

2. Връзки:

- Потребителят може да се регистрира, влиза и излиза.
- Пациентът прави запитвания и записва часове.
- Докторът принадлежи към отдел и е достъпен чрез времеви интервали.
- Отделението е към болница.
- Уреждането на среща се извършва от пациентите и включва лекари.
- Времевият интервал се управлява от лекари и показва достъпността.

4.2.3. ER Diagram (Entity–relationship model)



Фиг. 11 Модел на диаграма на Entity–relationship model

На Фиг. 11 е представен модел на ER диаграма (Entity-Relationship Diagram). Това е модел за визуално представяне на структурите на база данни, който показва как данните са организирани и как различните елементи на данните са свързани помежду си. ER диаграмите се използват основно в процеса на проектиране на бази данни, за да се идентифицират и описват отношенията между различните обекти (entities) в системата. Основните компоненти в този случай са:

1. Обекти:

- Patient (Пациент): Представява лица, получаващи медицински грижи.
- Enquiries (Запитвания): Обработка запитвания или въпроси от пациенти.

- Appointment (Среща): Представява планирани срещи.
- Time slot (Времеви интервал): Показва наличните часови интервали за срещи.
- Doctor (Доктор): Представява всички медицински специалисти.
- Department (Отдел): Отнася се за различни медицински отделения в рамките на болницата.
- Hospital (Болница): Представява цялостната институция.

2. Връзки:

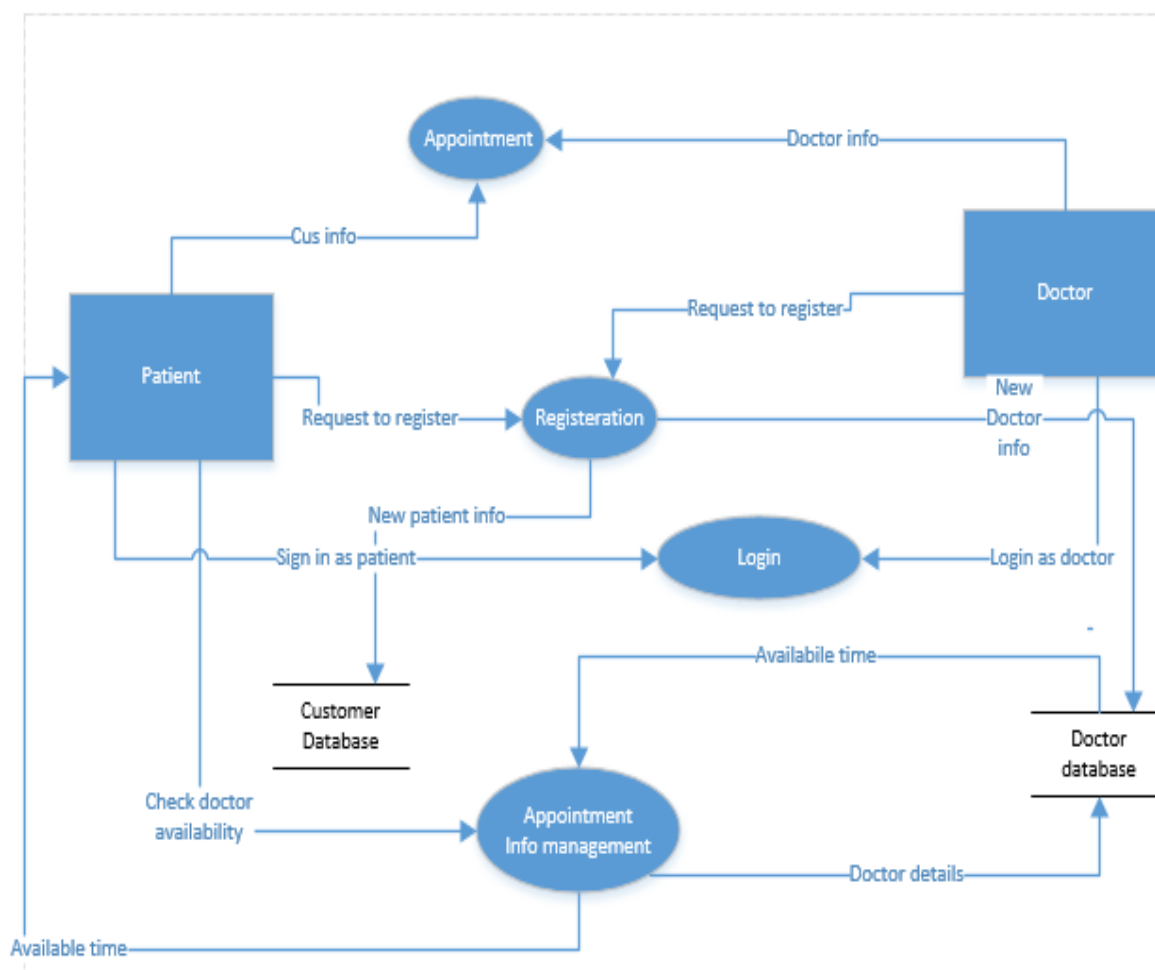
- makes (прави): Свързва пациентите с направените запитвания. Един пациент може да прави множество запитвания (1:n)
- books (запази): Свързва пациентите с уговорените срещи. Един пациент може да запази множество срещи с различни медицински специалисти (1:n)
- has (има): Свързва запазените срещи с медицинските специалисти. Един лекар може да има много уговорени срещи (1:n).
- available (има на разположение): Свързва времевите интервали с медицинските специалисти. Един лекар може да има множество времеви интервали на разположение.
- belongs to (принадлежи на): Свързва медицинските специалисти със съответните отделения. Един доктор може да принадлежи само към едно отделение (1:1).

3. Cardinality Indicators (Индикатори за кардиналност):

Индикаторите за кардиналност (Cardinality Indicators) са елементи в ER диаграмите, които показват броя на връзките между обектите (entities). Те определят колко инстанции на един обект могат да бъдат свързани с една инстанция на друг обект. Кардиналността е важен аспект при моделирането на бази данни, тъй като помага да се определи структурата на данните и техните взаимовръзки.

Кардиналността обикновено се показва чрез различни нотации, като например числа или символи до линиите, свързващи обектите. Числата (напр. „1“) показват колко екземпляра на един обект могат да бъдат свързани с друг.

4.2.4. Диаграма на потока от данни (Data Flow Diagram)



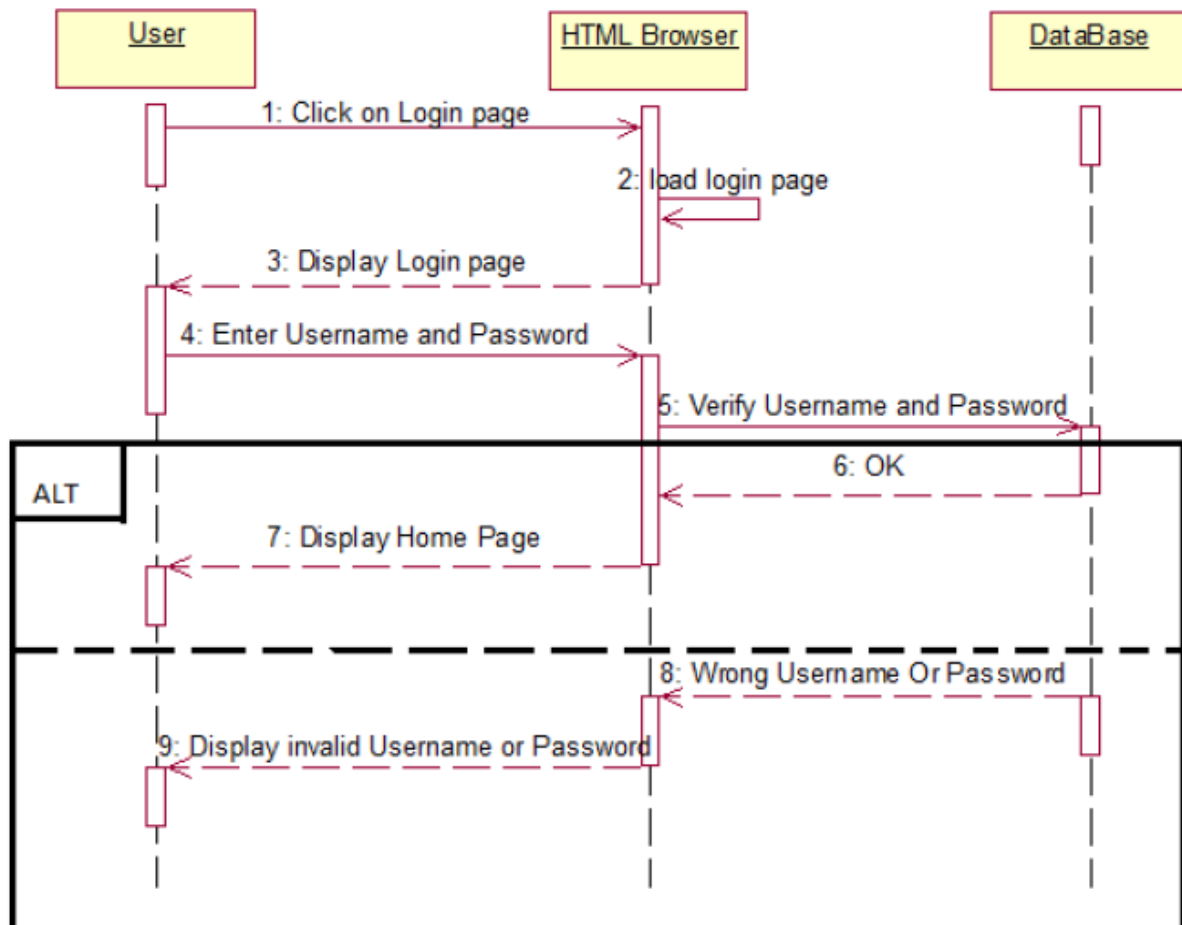
Фиг. 12 Модел на диаграма на потока от данни

На Фиг. 12 е представен модел на диаграма на потока от данни (Data Flow Diagram). Този вид диаграма се използва за визуализиране на потока на информация в рамките на системата или процеса. Някои от ключовите компоненти в случая са:

- **Процеси (Processes):** В диаграмата са изобразени различни процеси, които обработват информация. Например, “Appointment” (Записване на час), “Registration” (Регистрация), “Info management” (Управление на информация).
- **Хранилища на данни (Data Stores):** Това са местата, където се съхранява информацията. В случая има “Customer Database” (База данни на клиенти) и “Doctor Database” (База данни на лекари).
- **Външни субекти (External Entities):** Това са външни фактори, които влияят на системата. В случая се наблюдават “Patient” (Пациенти) и “Doctor” (Лекари).

- **Поток на данни (Data Flows):** Стрелките между компонентите показват потока на информация. Например, данните „Cus info“ от “Patient” (Пациент) се предават към “Appointment” (Записване на час).

4.2.5. Диаграма на последователност (Sequence Diagram)



Фиг. 13 Модел на диаграма на последователност за вход в системата

На Фиг. 13 е представен модел на диаграма на последователността (Sequence Diagram). Този вид диаграма се използва в софтуерното инженерство, за да покаже последователността на съобщенията, обменяни между различни обекти или компоненти в система, за да изпълнят функция или процес. В случая диаграмата включва три вертикални жизнени линии (lifelines), маркирани като “User” (Потребител), “HTML Browser” (Уеб браузър) и “DataBase” (База данни), които представят потока на взаимодействие за процеса на вход в системата.

Стъпки:

1. Потребителят натиска върху страницата за вход (login page).
2. Уеб браузърът зарежда и показва страницата за вход.
3. Потребителят въвежда потребителско име и парола.
4. Уеб браузърът изпраща тези данни за проверка.
5. Ако данните са невалидни, се изпраща съобщение за невалидно потребителско име или парола.
6. Ако данните са валидни, се изпраща съобщение “OK”, което води до показване на началната страница (home page).

Има и “ALT” рамка, която показва алтернативни сценарии в зависимост от това дали данните са валидни или не (стъпки 6-9). Ако са валидни, стъпки 6 и 7 водят до показване на началната страница, а ако не са, стъпки 8 и 9 водят до показване на съобщение за невалидно потребителско име или парола.

4.3. Приемни тестови случаи (Acceptance Test Cases)

4.3.1. Тест за приемане при влизане (Login Acceptance Test)

ID на тестовия случай:	ТС-01
Име на тестов случай:	Login (Доктор, Пациент, Администратор)
Критерии за успешно преминаване:	Тестът преминава, ако потребителят въведе правилно потребителско име или парола, които се съдържат в базата данни.
Входни данни:	Цифров и буквен ключов код
Тестова процедура:	Очаквани резултати:
<u>Стъпка 1</u> Въвеждане на неправилно потребителско име или парола	Системата издава звуков сигнал, за да покаже неизправност; записва неуспешен опит в базата данни; показва се съобщение за грешка, което информира потребителя за грешката.
<u>Стъпка 2</u> Въвеждане на правилно потребителско име и парола	Система показва успех; записва успешен достъп в базата данни; потребителят се пренасочва към началната страница
Забележка:	Страницата за вход има връзка, която насочва потребители, които все още не са регистрирани, към страницата за регистрация.

4.3.2. Тест за приемане на регистрация (Registration Acceptance Test)

ID на тестовия случай:	TC-02
Име на тестов случай:	Регистрация
Критерии за успешно преминаване:	Тестът преминава, ако потребителят предостави лична информация като име, имейл, парола, потвърждение на парола, дата на раждане, адрес, пол, телефон; цялата тази информация трябва да бъде въведена в съответните им полета във формуляра.
Входни данни:	Цифров и буквен ключов код
Тестова процедура:	Очаквани резултати:
<u>Стъпка 1</u> Празни полета на формуляра	Системата издава звуков сигнал, за да покаже неизправност; записва неуспешен опит в базата данни; показва се съобщение за грешка. Система показва успех; записва успешен достъп в базата данни. Ако паролата и потвърждението за паролата са еднакви, системата проверява дали имейлът вече е регистриран правилно, след което се създава нов потребител.
<u>Стъпка 2</u> Попълнени всички полета	
<u>Стъпка 3</u> Въведени различни пароли в полетата за парола и потвърждение за паролата	Неизправност на системата; записва се неуспешен опит в базата данни; показва се съобщение за грешка
<u>Стъпка 4</u> Въведени еднакви пароли в полетата за парола и потвърждение за паролата	Система показва успех; записва успешен достъп в базата данни; паролата и потвърждението за паролата са еднакви, след това системата проверява дали имейлът вече е регистриран правилно, след което се създава нов потребител.

4.3.3. Тест за приемане на началната страница (Home Page Acceptance Test)

ID на тестовия случай:	ТС-03
Име на тестов случай:	Начална страница
Критерии за успешно преминаване:	Тестът преминава, ако се отвори началната страница. Това е страницата по подразбиране на уебсайта, т.е. първата страница, която се вижда, когато се влезе в уебсайта или след регистрация.
Входни данни:	Въвеждане на името на сайта с връзка към интернет.
Тестова процедура:	Очаквани резултати:
<u>Стъпка 1</u> Въведено име или фамилия на лекар в полето за търсене	Система показва успех; записва успешен достъп в базата данни, потребителят може да търси лекари по име или фамилия; когато търсенето бъде изпратено, потребителят се пренасочва към друга страница и се показва списък с лекари, отговарящи на това описание.
<u>Стъпка 2</u> Въведено е нещо различно от име или фамилия на лекар в полето за търсене	Системата издава звуков сигнал, за да покаже неизправност; системата не може да намери никакви записи в базата данни и се показва съобщение за грешка.

4.3.4. Тест за приемане на резервация (Book Appointment Acceptance Test)

ID на тестовия случай:	ТС-04
Име на тестов случай:	Резервация на час
Критерии за успешно преминаване:	Тестът преминава, ако часът бъде запазен.
Входни данни:	Цифров и буквен ключов код
Тестова процедура:	Очаквани резултати:
<u>Стъпка 1</u> Избира се час, който вече е резервиран.	Системата показва „Недостъпно време“.

<u>Стъпка 2</u> Избира се свободен час	Системата запазва записа на срещата в базата данни; системата показва „Часът е резервиран!“
<u>Стъпка 3</u> Резервиране на час без вход в системата (login)	Потребителят се пренасочва към страницата за влизане, след което се пренасочва обратно към целевата/оригиналната страница.

4.3.5. Тест за приемане на потребителски профил (Account Profile Acceptance Test)

ID на тестовия случай:	ТС-05
Име на тестов случай:	Потребителски профил
Критерии за успешно преминаване:	Тестът преминава, ако потребителят кликне върху бутона за потребителския си профил и успешно влезе в него и успешно променя данните в него.
Входни данни:	Потребителят натиска бутон за влизане в редакциите на профила си; цифров и буквен ключов код
Тестова процедура:	Очаквани резултати:
<u>Стъпка 1</u> Редактиране на профил	Потребителят може да преглежда и редактира информация от профила; новата информация се записва и актуализира в базата данни.
<u>Стъпка 2</u> Преглед на срещите	Показва се информация за срещи с лекар, както настоящи, така и отминали такива.
<u>Стъпка 3</u> Редактиране на срещи	Потребителят може да променя датите на срещите; тази нова информация се актуализира и съхранява в базата данни.
<u>Стъпка 4</u> Отмяна на срещата	Осигурено е поле за референтен номер на резервация, чрез този номер резервацията на потребителя се анулира.
<u>Стъпка 5</u> Излизане от профила	Когато се щракне върху тази опция, потребителят излиза и се пренасочва към началната страница.

4.3.6. Тест за приемане на пациента (Patient Acceptance Test)

ID на тестовия случай:	ТС-06
Име на тестов случай:	Пациент
Критерии за успешно преминаване:	Тестът преминава, ако потребителят (администратор) щракне върху пациента. В този интерфейс са реализирани следните функционалности за администратора.
Входни данни:	Потребителят кликва и влиза в страницата на пациента.
Тестова процедура:	Очаквани резултати:
<u>Стъпка 1</u> Преглед на пациента	Показва се списък с всички пациенти в базата данни и когато се щракне върху пациентски елемент в списъка, подробностите за пациента се показват във формуляр.
<u>Стъпка 2</u> Добавяне на пациент	Администраторът може да добавя пациенти към базата данни, предоставя се формуляр и се въвеждат данните за пациента, когато формулярът бъде изпратен, новият пациент се създава и вмъква в базата данни.
<u>Стъпка 3</u> Редактиране на информацията за пациента	Администраторът може също така да редактира информацията за пациента, каквато и да е; въведената информация се записва и актуализира в базата данни.
<u>Стъпка 4</u> Изтриване на пациента от базата данни	Опция за изтриване се поставя до всеки пациент в списъка с пациенти и когато се щракне върху нея, конкретният пациент се изтрива.

4.3.7. Тест за приемане на час за среща (Appointment Acceptance Test)

ID на тестовия случай:	TC-07
Име на тестов случай:	Запазен час за среща
Критерии за успешно преминаване:	Тестът преминава, ако потребителят (администратор) щракне върху частта за запазен час за среща. В този интерфейс са реализирани следните функционалности за администратора.
Входни данни:	Потребителят кликва и влиза в страницата за редактиране на запазените часове за среща
Тестова процедура:	Очаквани резултати:
<u>Стъпка 1</u> Преглед на срещата	Показва се списък с всички срещи в базата данни и когато се щракне върху елемент от срещата в списъка, подробностите за срещата се показват във формуляр
<u>Стъпка 2</u> Добавяне на среща	Администраторът може да добавя срещи към базата данни, предоставя се формуляр и данните за срещата се въвеждат, когато формулярът бъде изпратен, новата среща се създава и вмъква в базата данни.
<u>Стъпка 3</u> Редактиране на информацията за срещата	Администраторът може също така да редактира информация за срещи, каквато и да е; въведената информация се запазва и актуализира в базата данни.
<u>Стъпка 4</u> Изтриване на срещата от базата данни	Опция за изтриване се поставя до всяка среща в списъка с срещи и когато се щракне върху нея, тази конкретна среща се изтрива.

4.3.8. Тест за приемане на запитванията (Make Enquiries Acceptance Test)

ID на тестовия случай:	ТС-08
Име на тестов случай:	Запитвания
Критерии за успешно преминаване:	Тестът е издържан, ако позволява на лекарят да преглежда и отговаря на запитвания, изпратени от пациентите.
Входни данни:	Цифров и буквен ключов код
Тестова процедура:	Очаквани резултати:
<u>Стъпка 1</u> Отговор на запитвания	Всички запитвания, изпратени от пациенти, се показват в списък с опция за отговор, предоставена за всяко запитване; когато се щракне върху него, се показва формуляр, където лекарят може да напише отговор на пациента; когато този формуляр бъде изпратен, съдържанието се изпраща като имейл към пациента.

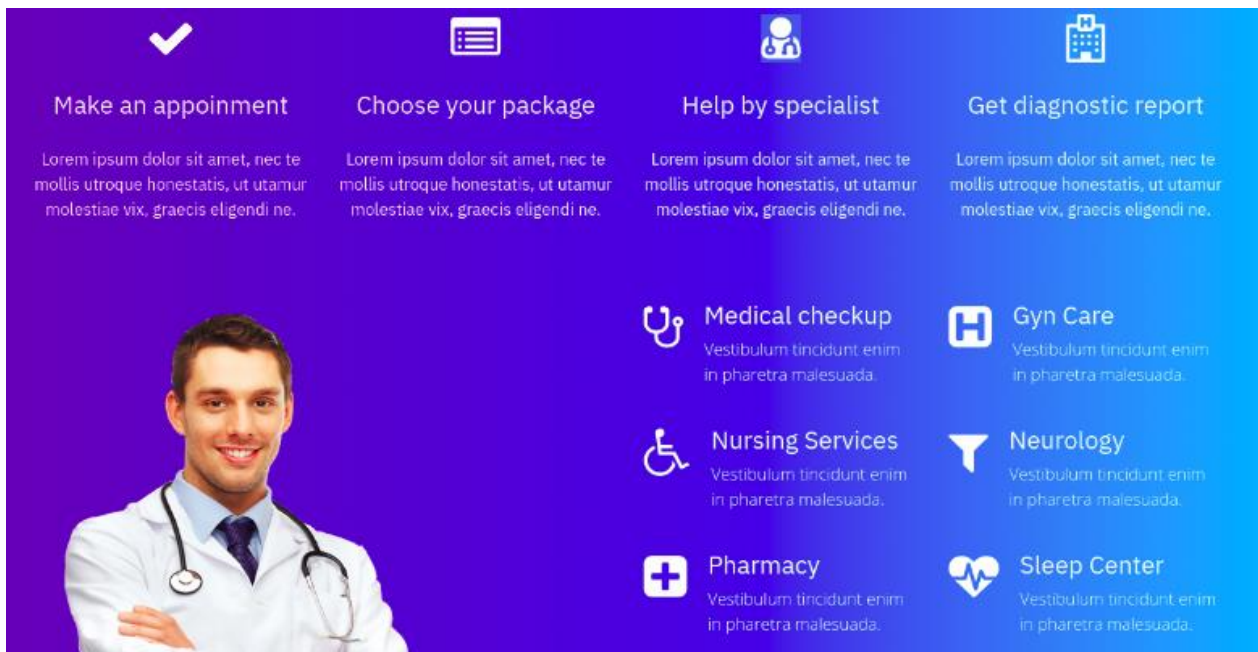
4.4. Примерен потребителски интерфейс

Тази система има „Начална страница“, откъдето пациентът, лекарят и администраторът могат да влязат в своите акаунти, като превключват съответно разделите. Фиг. 14 показва началната страница на този проект.

The screenshot displays the 'GLOBAL HOSPITALS' web application interface. The header features a navigation bar with 'HOME', 'ABOUT US', and 'CONTACT' links. Below the header, there is a user role selector with 'Patient', 'Doctor', and 'Admin' options. The main content area is titled 'Register as Patient' and contains a registration form with the following fields: 'First Name *', 'Last Name *', 'Your Email *', 'Your Phone *', 'Password *', and 'Confirm Password *'. There are also radio buttons for 'Male' and 'Female' selection, and a link for 'Already have an account?'. A 'Register' button is located at the bottom right of the form. On the left side of the page, there is a rocket icon and the word 'Welcome'.

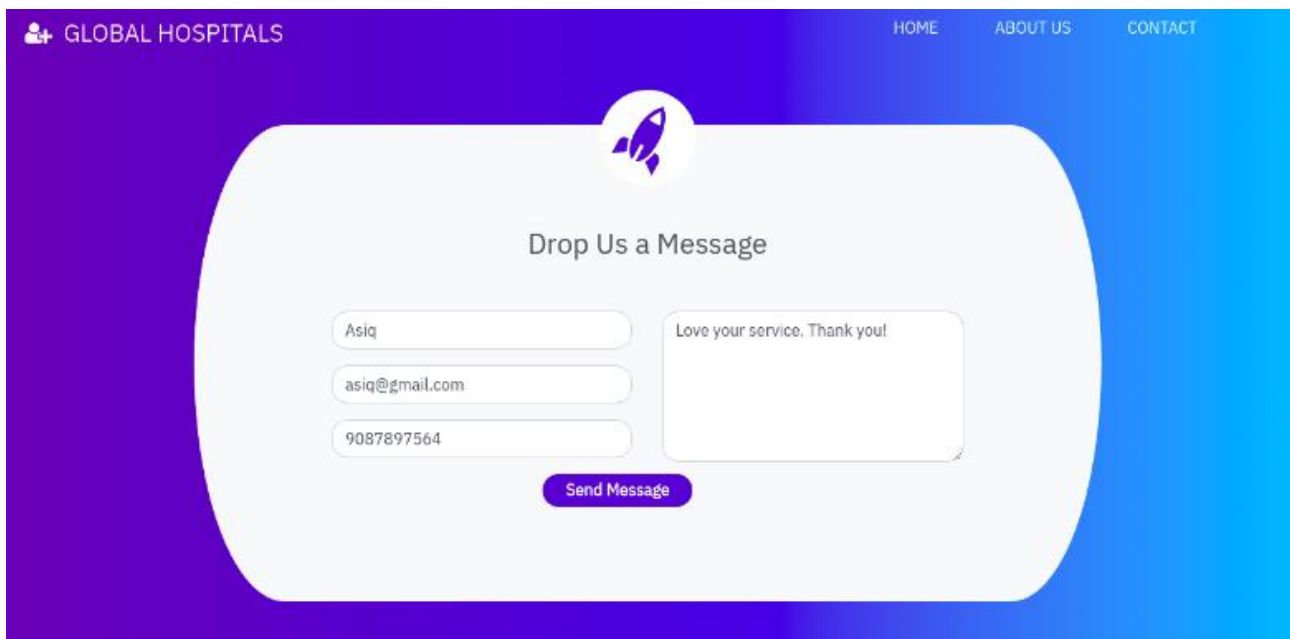
Фиг. 14 Начална страница на системата [48]

Страницата „За нас“ (Фиг. 15) ни позволява да получим малко повече информация за качеството и услугите на системата.



Фиг. 15 „За нас“ страница в процес на разработка [48]

Страницата „Контакти“ позволява на потребителите да предоставят обратна връзка или запитвания относно услугите на болницата. Фиг. 16 показва страницата „Контакти“.



Фиг. 16 Изглед от страницата за контакт [48]

„Началната“ страница се състои от 3 модула (Фиг. 14):

- Модул на пациента
- Докторски модул

- Администраторски модул

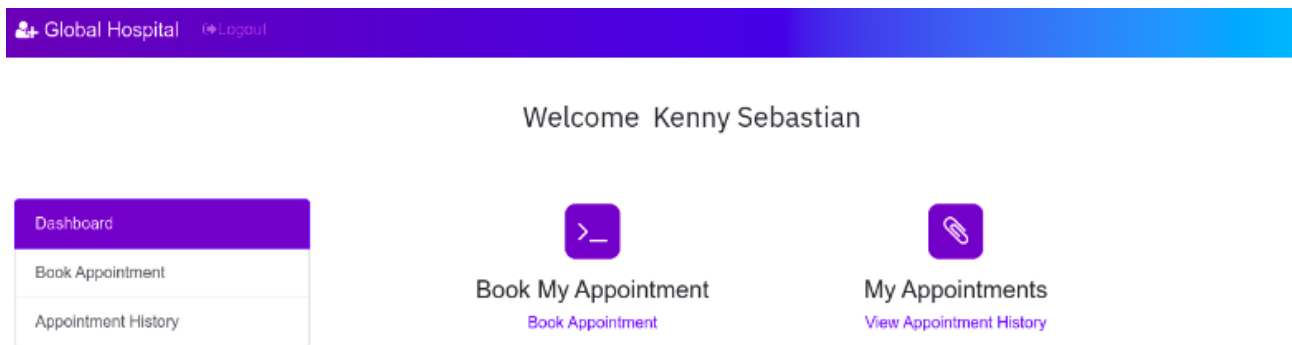
Модул на пациента:

Този модул позволява на пациентите да създадат свой акаунт, да си запишат час за преглед при лекар и да видят историята на своите срещи. Страницата за регистрация (в самата начална страница) изисква от пациентите да въведат своето собствено име, фамилия, имейл ID, номер за контакт, парола и пол.

След като пациентът е създал свой собствен акаунт след щракване върху бутона „Регистрация“, той ще бъде пренасочен към своето табло за управление (Фиг. 18).

The screenshot shows the 'Register as Patient' form on the GLOBAL HOSPITALS website. The page has a purple header with the logo and navigation links (HOME, ABOUT US, CONTACT). On the left, there is a 'Welcome' message with a rocket icon. The registration form is on the right, with tabs for 'Patient', 'Doctor', and 'Admin'. The form fields include: First Name (Kenny), Last Name (Sebastian), Email (kenny@gmail.com), Phone Number (8797868769), Password (masked with asterisks), and Gender (Male selected, Female unselected). A 'Matching' status is shown next to the password field. There is a link 'Already have an account?' and a 'Register' button.

Фиг. 17 Изглед от регистрацията като пациент [48]



Фиг. 18 Изглед от таблото за управление на пациента [48]

Страницата на таблото за управление позволява на пациентите да извършат две операции:

1. Запазване на час:

Тук пациентите могат да запишат час за лекар. Формулярът за записване (Фиг. 19) изисква пациентите да изберат лекаря, когото искат да видят, датата и часа, когато искат да се срещнат с лекаря. Таксата за консултация ще бъде показана съответно на пациента, както вече е определена от лекаря.

Global Hospital Logout

Welcome Kenny Sebastian

Dashboard
Book Appointment
Appointment History

Create an appointment

Doctors: Ganesh

Consultancy Fees 550

Date 10/10/2019

Time 10:00

Create new entry

Фиг. 19 Изглед от формуляра за записване на час [48]

След щракване върху бутона „Създаване на нов запис“, пациентът ще получи предупреждение, което потвърждава успешното запазване на часа за преглед при медицински специалист. (Фиг. 20).

Global Hospital Logout

localhost says
Your appointment successfully booked
OK

Dashboard
Book Appointment
Appointment History

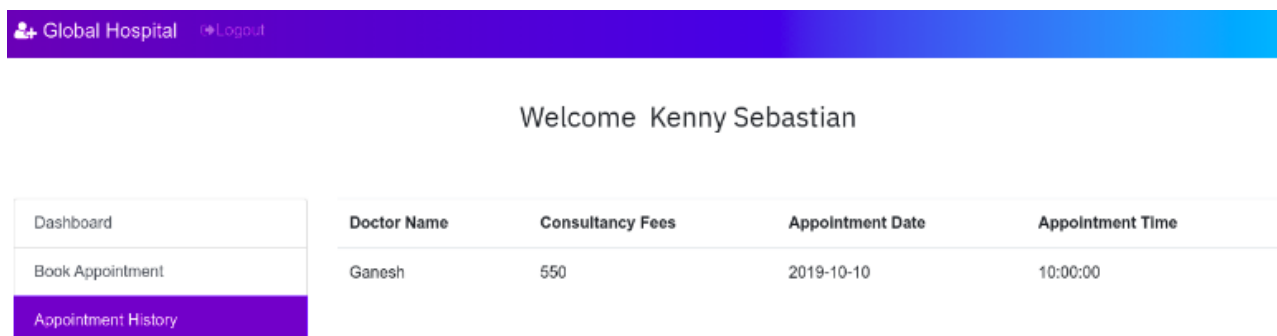
Book My Appointment
Book Appointment

My Appointments
View Appointment History

Фиг. 20 Потвърждение за успешно резервиране на час [48]

2. Преглед на хронологията на срещите с лекар:

Тук пациентът може да види хронологията на своите срещи, която съдържа име на лекар, такса за консултация, дата и час на среща. (Фиг. 21).

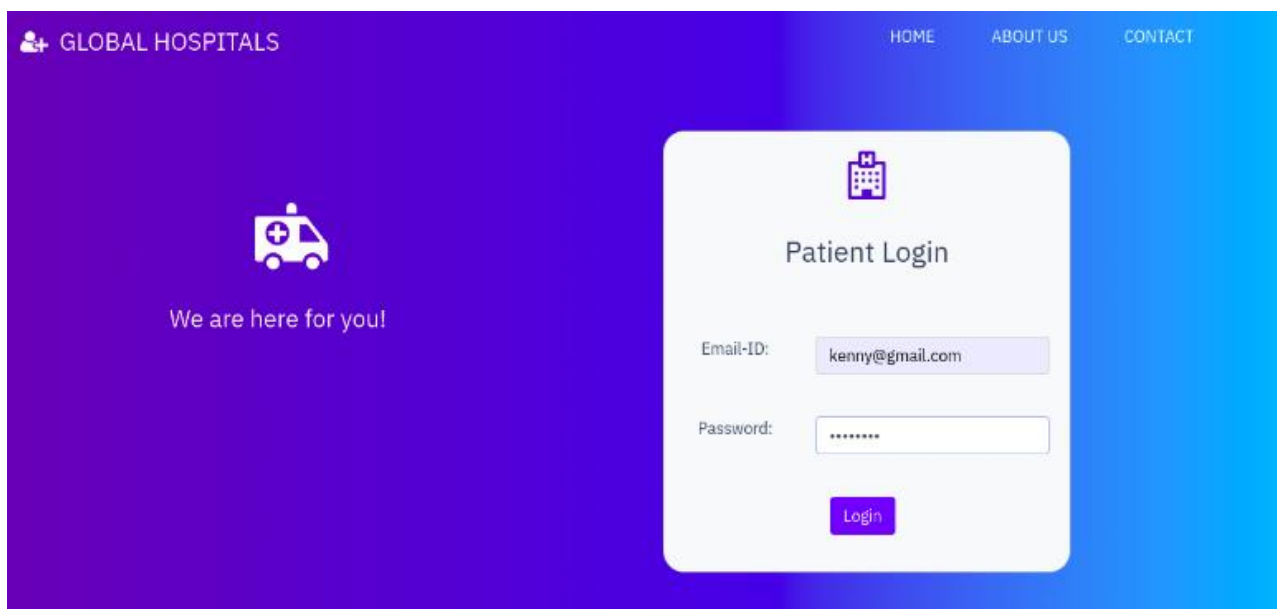


The screenshot shows a user interface for a hospital's appointment system. At the top, a purple header bar contains the text "Global Hospital" and a "Logout" link. Below the header, a welcome message "Welcome Kenny Sebastian" is displayed. On the left, a vertical sidebar menu lists three options: "Dashboard", "Book Appointment", and "Appointment History", with the latter highlighted in purple. To the right of the sidebar is a table with four columns: "Doctor Name", "Consultancy Fees", "Appointment Date", and "Appointment Time". The table contains one row of data for a doctor named Ganesh, with a fee of 550, an appointment date of 2019-10-10, and a time of 10:00:00.

Doctor Name	Consultancy Fees	Appointment Date	Appointment Time
Ganesh	550	2019-10-10	10:00:00

Фиг. 21 Изглед от модула за история на срещите [48]

След като пациентът излезе от акаунта си, ако иска да влезе отново в профила си, той може да влезе в него, вместо да се регистрира отново. Фиг. 22 показва страницата за вход. Щракването върху бутона „Вход“ ще пренасочи пациента към страницата на неговото табло, която беше разгледана по-рано (Фиг. 18).



The screenshot displays the patient login page of the Global Hospitals website. The page has a purple header with the "GLOBAL HOSPITALS" logo and navigation links for "HOME", "ABOUT US", and "CONTACT". The main content area features a large purple banner with a white ambulance icon and the text "We are here for you!". To the right, a white login box titled "Patient Login" contains input fields for "Email-ID:" (with the value "kenny@gmail.com") and "Password:" (masked with asterisks). A purple "Login" button is positioned below the password field.

Фиг. 22 Изглед от страницата за вход като пациент [48]

5. Заключение

Дипломната работа представя система, която цели да улесни достъпа до медицински консултации чрез интернет. Проектът включва детайлно анализиране на софтуерни изисквания, проектиране на архитектурата и разработка на функционалности, които са съобразени със специфичните нужди на потребителите и медицинските специалисти.

Изготвените UML диаграми играят ключова роля в разработката на системата за онлайн консултации с медицински специалисти. Те предоставят ясно представяне на структурата и взаимодействията между различните компоненти на системата. Анализът на диаграмите показва оптимизирани процеси за комуникация и данни, осигуряващи ефективност и устойчивост на системата.

Детайлните спецификации на хардуера, софтуера и интерфейсите предоставят основа за успешната интеграция и функциониране на системата. Проектираните хардуерни и софтуерни компоненти, както и интерфейсните стандарти, осигуряват надеждна работа на платформата за консултации, съчетавайки сигурност и удобство за потребителите.

Резултатите от извършените приемни тестове демонстрират висока степен на функционална и сигурностна стабилност на системата. Всички ключови функционалности, включително защитата на данните на потребителите и надеждността на връзките, са уверено валидирани и тествани, осигурявайки безопасност и доверие за крайните потребители.

Проектираният потребителски интерфейс предлага интуитивно и лесно използване за потребителите. Ергономичният дизайн и логическата организация на интерфейсните елементи подобряват потребителското изживяване и способстват за активната ангажираност на потребителите със системата за онлайн консултации.

За по-нататъшно развитие на научните и техническите аспекти, свързани със системите за онлайн консултации, могат да се вземат предвид следните препоръки:

1. Разширяване на функционалностите за анализ и прогнозиране на данни:

Интегрирането на напреднали алгоритми за анализ на данни и извличане на знания може да подобри качеството на предоставяните медицински консултации.

2. Разработване на мобилни приложения и подобрени мобилни интерфейси:

Осигуряването на мобилни решения ще увеличи достъпността и удобството за потребителите, които предпочитат да използват смартфони и таблети за достъп до системата.

3. Интеграция на виртуална реалност и разширена реалност: Използването на AR и VR технологии може да предложи нови начини за интерактивност между пациенти и лекари, като симулира реални медицински среди и улеснява обучението и диагностиката.

4. Усъвършенстване на алгоритмите за сигурност и защита на данните: С разрастването на онлайн платформите за здравеопазване, е от съществено значение да се усъвършенства защитата на личните данни и комуникациите със сигурностни мерки на високо ниво.

Тези препоръки не само ще укрепят текущата система, но също така ще подпомогнат разширяването ѝ към нови възможности и разработки в областта на телемедицината и софтуерните технологии.

Литература

- [1] “Телемедицина – същност, история, предизвикателства”, Consentо (05.05.2019), <https://consento.bg/telemedicine-opredelenie/>
- [2] “The Role of Telehealth in an Evolving Health Care Environment: Workshop Summary.”, Board on Health Care Services, 2012 Nov 20. - <https://www.ncbi.nlm.nih.gov/books/NBK207141/>
- [3] “Understanding Telehealth Regulations and Requirements”, Terrey L. Hatcher, July 2, 2020 - <https://www.relias.com/blog/understanding-telehealth-regulations>
- [4] “TELEMEDICINE: USING TELECOMMUNICATION INFRASTRUCTURE TO RESPOND TO GROWING CHALLENGES IN THE HEALTH SYSTEM”, Assoc. Prof. PhD Dimitar Minchev, Petar Trashliev, Списание „Компютърни науки и комуникации”, Том 10, No1 (2021), БСУ, Бургас - https://www.google.com/url?sa=t&rct=j&q=&esrc=s&source=web&cd=&cad=rja&uact=8&ved=2ahUKewiArKm384mHAxXuRPEDHT3aCKsQFnoECBYQAQ&url=https%3A%2F%2Fcsc.bfu.bg%2Findex.php%2FCSC%2Farticle%2Fview%2F254%2F254&usg=AOvVaw0yPY81v6s2YPZASmsR_vBm&opi=89978449
- [5] “Разработка на софтуер - етапи, нужни специалисти и още”, Диляна Койчева, 23 окт 2023, <https://advanceacademy.bg/blog/razrabotka-na-software-etapi-nuzhni-specialisti-i-oshte>
- [6] “SDLC - Waterfall Model” - https://www.tutorialspoint.com/sdlc/sdlc_waterfall_model.htm
- [7] “SDLC – Prototyping Model”, Notepub (Official) / April 2, 2021 - <https://notepub.io/notes/software-engineering/software-development-life-cycle/sdlc-prototyping-model/>
- [8] “Spiral Model”, Kuldeep Rana, April 28, 2021 - <https://artoftesting.com/spiral-model>
- [9] “SDLC – V-Model” - https://www.tutorialspoint.com/sdlc/sdlc_v_model.htm
- [10] “The Evolution of Agile: Implementing SCRUM in Software Engineering”, Nacho García, Oct 8, 2023 - <https://medium.com/@nachomnl/the-evolution-of-agile-implementing-scrum-in-software-engineering-98feb9dc04cb>
- [11] “What is Feature Driven Development (FDD)? An Overview”, Marjan Venema, April 29, 2024 - <https://www.nimblework.com/agile/feature-driven-development-fdd/>
- [12] “The Kanban Method: The Ultimate Beginner’s Guide!”, Sonya Siderova - <https://getnave.com/blog/what-is-the-kanban-method/>

- [13] “How to Write an SRS Document (Software Requirements Specification Document)”, Gerhard Krüger and Charles Lane, January 17, 2023 - <https://www.perforce.com/blog/alm/how-write-software-requirements-specification-srs-document>
- [14] “11 Requirements Gathering Techniques for Agile Product Teams” - <https://www.jamasoftware.com/requirements-management-guide/requirements-gathering-and-management-processes/11-requirements-gathering-techniques-for-agile-product-teams>
- [15] “HIPAA vs. GDPR compliance: what’s the difference?”, September 21, 2022 - <https://www.onetrust.com/blog/hipaa-vs-gdpr-compliance/>
- [16] “Кои са съвременните технологии за защита на информацията?”, апр. 29, 2024 - <https://startupfactory.bg/blogbg/technologii-za-kibezashrita/>
- [17] “15 Most Important Usability Principles for User Interface Design”, Mansurul Haque, Feb 8, 2023 - <https://medium.com/@mdmansurulhaques/15-important-usability-principles-for-user-interface-design-ea819cedeb36>
- [18] “8 Essential usability testing methods for actionable UX insights” - <https://maze.co/guides/usability-testing/methods/>
- [19] “Contextual Inquiry: Inspire Design by Observing and Interviewing Users in Their Context”, Kim Salazar, December 6, 2020 - <https://www.nngroup.com/articles/contextual-inquiry/>
- [20] “Journey Mapping 101”, Sarah Gibbons, December 9, 2018 - <https://www.nngroup.com/articles/journey-mapping-101/>
- [21] “Diary Studies: Understanding Long-Term User Behavior and Experiences”, Kim Salazar, March 29, 2024 - <https://www.nngroup.com/articles/diary-studies/>
- [22] “Usability Metrics”, Jakob Nielsen, January 20, 2001 - <https://www.nngroup.com/articles/usability-metrics/>
- [23] “Putting A/B Testing in Its Place”, Jakob Nielsen, August 14, 2005 - <https://www.nngroup.com/articles/putting-ab-testing-in-its-place/>
- [24] “EHR systems – electronic health records explained”, Liam Sheasby - <https://www.theaccessgroup.com/en-gb/health-social-care/software/electronic-patient-records/ehr-systems-electronic-health-records-explained/>
- [25] “A detailed view of Hospital Management System (HMS)”, Sanjana, 04-10-2018 - <https://mocdoc.com/blog/a-detailed-view-of-hospital-management-system-hms>
- [26] “Electronic prescription system requirements: a scoping review”; Marjan Vejdani, Mehdi Varmaghani, Marziyhe Meraji, Jamshid Jamali, Elaheh Hooshmand and Ali Vafae-Najar; 2022 Sep 3; <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9440517/>

- [27] “Laboratory information system (LIS)”, Gavin Wright - <https://www.techtarget.com/searchhealthit/definition/laboratory-information-system>
- [28] “10 Key Aspects of Interoperability in Healthcare Everyone Should Know”; Kara L. Nadeau; Wednesday, April 12, 2023 - <https://www.ghx.com/the-healthcare-hub/healthcare-interoperability-guide/>
- [29] “Healthcare Standards Overview: HL7, FHIR, DICOM, CCD, ICD-9, ICD-10”; On 2019-07-08; - <https://www.covetus.com/blog/healthcare-standards-overview-hl7-fhir-dicom-ccd-icd-9-icd-10>
- [30] “What is an API (application programming interface)?”, Michael Goodwin, 09 April 2024 - <https://www.ibm.com/topics/api>
- [31] “Why is Java ‘write once and run anywhere?’”, Last Updated : 26 May, 2019 - <https://www.geeksforgeeks.org/why-is-java-write-once-and-run-anywhere/>
- [32] “Java garbage collection: What is it and how does it work?”, Kanak Rana, Last Updated: Mar 27, 2024 - <https://www.naukri.com/code360/library/what-is-garbage-collection-in-the-context-of-java>
- [33] “What is Spring Framework and Hibernate ORM?”, Last Updated : 12 May, 2023 - <https://www.geeksforgeeks.org/what-is-spring-framework-and-hibernate-orm/>
- [34] “Multithreading in Java”, Last Updated : 24 Feb, 2021 - <https://www.geeksforgeeks.org/multithreading-in-java/>
- [35] “Decoding the Secrets of Top Programming Languages: JavaScript, Python, Java, C#, and C++”, The Keyboard Wordsmith, Jan 11, 2023 - <https://medium.com/@keyboardwordsmith/decoding-the-secrets-of-top-programming-languages-javascript-python-java-c-and-c-8b184f966bb7>
- [36] “Relational vs. Non-Relational Databases”, Tamara Pattinson, Updated on November 9, 2022 - <https://www.pluralsight.com/blog/software-development/relational-vs-non-relational-databases>
- [37] “ACID Properties in DBMS”, Last Updated : 25 Jun, 2024 - <https://www.geeksforgeeks.org/acid-properties-in-dbms/>
- [38] “What is MySQL?” - <https://www.oracle.com/mysql/what-is-mysql/>
- [39] “Understanding MVC (Model-View-Controller) Architecture”, Elif İrem Kara Kadyrov, Aug 21, 2023 - <https://mobileappcircular.com/understanding-mvc-model-view-controller-architecture-a2208d924b2>
- [40] “What is Unified Modeling Language (UML)?” - <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-uml/>
- [41] “What is Use Case Diagram?” - <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-use-case-diagram/>

- [42] “What is Class Diagram?” - <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-class-diagram/>
- [43] “Introduction of ER Model”, Last Updated : 03 Jul, 2024 - <https://www.geeksforgeeks.org/introduction-of-er-model/>
- [44] “What is a Data Flow Diagram” - <https://www.lucidchart.com/pages/data-flow-diagram>
- [45] “What is Sequence Diagram?” - <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-sequence-diagram/>
- [46] “Acceptance Testing : What, Why, Types & How to Do?” - <https://testsigma.com/guides/acceptance-testing/>
- [47] “What is Use Case Specification?” - <https://www.visual-paradigm.com/guide/use-case/what-is-use-case-specification/>
- [48] <https://github.com/Aryan-08/hospital-management-system>

Списък на фигурите

Фиг. 1 Схема на модела на водопада [6].....	7
Фиг. 2 Схема на прототипен модел [7]	9
Фиг. 3 Схема на спирален модел [8]	10
Фиг. 4 Схема на V-образен модел [9].....	11
Фиг. 5 Схема на Scrum модела [10]	12
Фиг. 6 Схема на FDD модела [11]	15
Фиг. 7 Схема на Канбан дъска [12].....	16
Фиг. 8 Общ модел на MVC архитектура [39]	41
Фиг. 9 Модел на диаграма на случаите на използване за системата	62
Фиг. 10 Модел на диаграма на класовете за системата.....	64
Фиг. 11 Модел на диаграма на Entity–relationship model	67
Фиг. 12 Модел на диаграма на потока от данни	69
Фиг. 13 Модел на диаграма на последователност за вход в системата	70
Фиг. 14 Начална страница на системата [48]	79
Фиг. 15 „За нас“ страница в процес на разработка [48].....	80
Фиг. 16 Изглед от страницата за контакт [48].....	80
Фиг. 17 Изглед от регистрацията като пациент [48]	81
Фиг. 18 Изглед от таблото за управление на пациента [48]	82
Фиг. 19 Изглед от формуляра за записване на час [48]	83
Фиг. 20 Потвърждение за успешно резервиране на час [48]	83
Фиг. 21 Изглед от модула за история на срещите [48]	84
Фиг. 22 Изглед от страницата за вход като пациент [48]	84

Списък на таблиците

Таблица 1 Спецификация на случаите на употреба за вход (login) в системата.....	53
Таблица 2 Спецификация на случаите на употреба за регистрация в системата	54
Таблица 3 Спецификация на случаите на употреба за резервация на час за преглед	55
Таблица 4 Спецификация на случаите на употреба за преглед и промяна на данните	56
Таблица 5 Спецификация на случаите на употреба за задаване на свободно време.....	57
Таблица 6 Спецификация на случаите на употреба за отмяна на записан час	58
Таблица 7 Спецификация на случаите на употреба за търсене на лекар.....	59
Таблица 8 Спецификация на случаите на употреба за управление на потребителите.....	60
Таблица 9 Спецификация на случаите на употреба за управление и достъп до базата	61